

GPU-Accelerated Isoalgorítmico GA+2-opt para TSP

Lucas Galdino

2025-11-22

Lista de Símbolos

Símbolo	Descrição
n_{coords}	Número de cidades (coordenadas) da instância
n_{pop}	Tamanho da população do Algoritmo Genético
n_{reps}	Número de repetições independentes por experimento
p_{mut}	Taxa de mutação
p_c	Taxa de cruzamento
k_{trnmt}	Tamanho do torneio na seleção
n_{2-opt}	Número máximo de iterações da busca local 2-opt
B_{H2D}	Volume de dados transferidos Host-to-Device
B_{D2H}	Volume de dados transferidos Device-to-Host
T	Tempo de execução
ΔC	Variação de custo em um movimento local
$G = (V, E)$	Grafo composto por vértices V e arestas E
c_{ij}	Custo (distância) entre as cidades i e j
$f(x)$	Função de aptidão (<i>fitness</i>) de uma solução x
$P(t)$	População de soluções na geração t
κ	Límite de iterações para busca local

Lista de Figuras

5	Arquitetura de CPU vs GPU. Fonte: LayerStack.	26
6	Alocação de silício em CPU e GPU. (30)	26
7	Organização interna de GPU moderna. (31)	27

Lista de Tabelas

2	Instâncias da TSPLIB selecionadas para o estudo	34
---	---	----

3	Resultados detalhados por instância (Seleção)	38
4	Médias Agregadas (Todas as Instâncias)	39
5	Melhor Algoritmo (Menor Gap) por Tamanho	40

Resumo

Este trabalho investiga o impacto de diferentes estratégias de paralelização em Unidades de Processamento Gráfico (GPUs) no desempenho de um algoritmo memético (Algoritmo Genético híbrido com busca local 2-opt) para o Problema do Caixeiro Viajante (TSP). Foram implementadas e comparadas quatro variantes isoalgorítmicas: uma versão sequencial em CPU, uma versão híbrida ingênua (transferência indivíduo a indivíduo), uma versão híbrida otimizada (processamento em lote) e uma versão totalmente residente em GPU. Os experimentos, conduzidos em um conjunto de 38 instâncias da TSPLIB com até 1002 cidades, demonstram que a estratégia de paralelização influencia drasticamente o trade-off entre tempo de execução e qualidade da solução. A variante híbrida otimizada obteve os menores tempos de execução (Ganho de Performance médio de 5.5x sobre a híbrida ingênua), enquanto a variante totalmente em GPU alcançou a melhor qualidade de solução (gap médio de 0.88%). Os resultados evidenciam que a minimização da transferência de dados entre CPU e GPU é crítica para o desempenho, e que arquiteturas totalmente residentes favorecem o refinamento das soluções em problemas de otimização combinatória.

Palavras-chave: Problema do Caixeiro Viajante, Algoritmos Genéticos, Computação em GPU, CUDA, Otimização Combinatória.

Abstract

This work investigates the impact of different parallelization strategies on Graphics Processing Units (GPUs) on the performance of a memetic algorithm (Genetic Algorithm hybridized with 2-opt local search) for the Traveling Salesman Problem (TSP). Four iso-algorithmic variants were implemented and compared: a sequential CPU version, a naive hybrid version (individual-by-individual transfer), an optimized hybrid version (batch processing), and a fully GPU-resident version. Experiments conducted on a set of 38 TSPLIB instances with up to 1002 cities demonstrate that the parallelization strategy drastically influences the trade-off between execution time and solution quality. The optimized hybrid variant achieved the lowest execution times (average Ganho de Performance of 5.5x over the naive hybrid), while the fully GPU-resident variant achieved the best solution quality (average gap of 0.88%). The results highlight that minimizing data transfer between CPU and GPU is critical for performance, and that fully resident architectures favor solution refinement in combinatorial optimization problems.

Keywords: Traveling Salesman Problem, Genetic Algorithms, GPU Computing,

CUDA, Combinatorial Optimization.

- Lista de Símbolos
- Resumo
- Abstract
- Capítulo 1 – Introdução
 - 1.1 Contexto e motivação
 - 1.2 Problema de pesquisa
 - 1.3 Objetivo geral
 - 1.4 Objetivos específicos
 - 1.5 Justificativa
 - 1.6 Organização do trabalho
- Capítulo 2 - Revisão bibliográfica
 - 2. Fundamentação teórica e revisão bibliográfica
 - * 2.1 Otimização combinatória e o Problema do Caixeiro Viajante
 - * 2.2 Heurísticas para o TSP
 - 2.2.1 Entendendo o algoritmo 2-opt
 - * 2.3 Algoritmos genéticos
 - * 2.4 Heurísticas híbridas
 - * 2.5 Algoritmos meméticos
 - 2.5.1 Algoritmo Genético + 2-opt
 - * 2.6 Computação em GPU e paralelização de meta-heurísticas
 - 2.6.1 Taxonomia de paralelização segundo Crainic e Toulouse
 - 2.6.2 Aplicações de GPU a meta-heurísticas para o TSP
 - 2.6.3 Contraste arquitetural: CPU vs GPU
 - * 2.7 Comparação estatística de algoritmos de otimização
 - 2.7.1 Comparações pareadas
 - 2.7.2 Comparações múltiplas
- Capítulo 3 – Materiais e Métodos
 - 3.1 Ambiente computacional e framework experimental
 - * 3.1.1 Hardware e sistema operacional
 - * 3.1.2 Ambiente de software
 - * 3.1.3 Arquitetura do framework experimental
 - 3.2 Arquitetura do framework e variantes algorítmicas
 - * 3.2.1 O Algoritmo Genético Base (GA + 2-opt)
 - * 3.2.2 Estratégias de Paralelização (As 4 Variantes)
 - 3.3 Seleção de instâncias e protocolo experimental
 - 3.4 Protocolo de análise estatística
- Capítulo 4 – Resultados
 - 4.1 Resultados Experimentais
 - 4.2 Análise por Instância
 - 4.3 Análise Agregada e Trade-offs
 - 4.4 Melhor Algoritmo por Categoria de Tamanho
 - 4.5 Discussão Geral

Capítulo 1 – Introdução

1.1 Contexto e motivação

O Problema do Caixeiro Viajante (Traveling Salesman Problem – TSP) é um dos problemas mais estudados em otimização combinatória (1). Tem esse nome devido à sua história clássica: um vendedor precisa visitar um conjunto de cidades, passando por cada uma exatamente uma vez, e deseja minimizar a distância total percorrida.

O objetivo é encontrar um ciclo hamiltoniano (ciclo no qual se deve passar por todos os vértices uma vez e retornar ao vértice original). Apesar de sua formulação simples, o TSP é NP-difícil (classe de problemas para os quais não se conhece algoritmo polinomial eficiente) e, na sua formulação de decisão: “existe um tour com custo menor ou igual a B ?”, é NP-completo: não se conhece algoritmo em tempo polinomial que o resolva em geral, embora seja fácil verificar o custo de uma solução candidata. Cook (1) argumenta que essa combinação de simplicidade, dificuldade teórica e rica estrutura geométrica faz do TSP um estudo de caso central tanto para a teoria da complexidade quanto para desenvolvimento de métodos exatos e heurísticos.

Na prática, variantes do TSP aparecem em domínios como roteirização de veículos, planejamento de inspeções, manufatura e testes de circuitos, genômica e astronomia, entre outros (1). Exemplos clássicos incluem o posicionamento e ordenamento de furos em placas de circuito impresso, logística e problemas de mapeamento genético. Mesmo quando modelos reais são mais complexos (com janelas de tempo, múltiplos veículos ou restrições de capacidade), é comum validar ideias de projeto e análise de algoritmos primeiro em instâncias clássicas do TSP, justamente pela ampla disponibilidade de referências, testes padronizados e de soluções ótimas conhecidas.

Paralelamente, a evolução do hardware trouxe processadores gráficos (GPUs) como plataforma acessível para computação de alto desempenho (HPC) (2) e com a popularização e acessibilidade, pesquisas nestas áreas cresceram rapidamente. Modelos de Inteligência Artificial Generativa, por exemplo, dependem fortemente desse tipo de hardware.

GPUs oferecem milhares de núcleos relativamente simples, organizados em um modelo de execução massivamente paralelo, mais próximo do paradigma SIMD/SIMT descrito por Flynn e extensões modernas (3). Em troca de um controle mais restrito nos fluxos de processo e memória, essas arquiteturas entregam uma grande largura de banda de memória (comunicação entre CPUs e GPUs) e uma grande taxa de operações aritméticas por segundo (FLOPs), desde que o problema ofereça operações semelhantes que possam ser executadas em paralelo, seja utilizando paralelismo funcional (mais complexo) ou paralelismo de dados (mais simples). Isso torna GPUs particularmente atrativas para tarefas como o cálculo de matrizes (ganho exponencial, como demonstrado nesse trabalho), a avaliação de grandes populações de soluções e a aplicação de

movimentos de vizinhança independentes em heurísticas de melhoria.

Do ponto de vista das meta-heurísticas, Crainic e Toulouse propõem uma taxonomia de paralelização que distingue estratégias baseadas em decomposição de dados, decomposição de domínio e múltiplas trajetórias cooperativas (4). A discussão detalhada dessas categorias e sua adequação a arquiteturas de GPU é apresentada na Seção 2.6.1.

O projeto desta monografia foca principalmente em algoritmos do tipo 1, explorando paralelismo de dados em vizinhanças de 2-opt e na avaliação de populações em algoritmos genéticos, em linha com estudos recentes sobre heurísticas para TSP em GPU (5,6,7).

Ainda assim, nem todo algoritmo se beneficia automaticamente de uma migração para GPU. Inicializações de kernels, movimentação de dados entre CPU e GPU podem anular ganhos de paralelismo, especialmente em problemas de porte pequeno ou em implementações que realizam pouco trabalho por inicializações de kernel (8,9). Limitações de memória (VRAM) devem ser cuidadosamente calculadas para que a memória total ocupada não passe do limite do hardware. Nesses cenários, uma comparação direta entre versões em CPU e GPU exige cuidado extra: deve-se isolar o impacto da plataforma de execução sem confundir o efeito com mudanças na lógica do algoritmo. Ao longo deste trabalho, esses desafios são documentados explicitamente em estudos de caso, onde kernels são chamados de forma não cautelosa e versões otimizadas (Capítulos 3 e 4), bem como em notas técnicas específicas sobre *overhead* de lançamentos de kernel e padrões de redução em GPU.

Este trabalho insere-se nesse contexto, investigando como acelerar, utilizando GPU, um Algoritmo Genético (Genetic Algorithm – GA) híbrido com 2-opt para o TSP, de forma controlada e estatisticamente fundamentada, usando instâncias clássicas da biblioteca TSPLIB (10) e um protocolo experimental com um número adequado de repetições n_{reps} e testes estatísticos apropriados (detalhado no Capítulo 3). A combinação entre GA e busca local é um exemplo de algoritmo memético amplamente estudado na literatura (11,12), em que operadores evolutivos globais são complementados por heurísticas de melhoria como 2-opt (13) ou Lin–Kernighan (14). Neste Trabalho de Conclusão de Curso, usarei a nomenclatura em inglês para alguns termos técnicos como “2-opt”, “TSPLIB”, “GPU”, “CPU” e “GA”.

1.2 Problema de pesquisa

Muitos estudos em computação de alto desempenho comparam algoritmos em CPU e GPU utilizando implementações que diferem não apenas na plataforma de execução, mas também em detalhes relevantes da lógica do algoritmo (por exemplo, operadores distintos, parâmetros diferentes ou vizinhanças não equivalentes) (5,8,15). Isso torna difícil atribuir ganhos de desempenho exclusivamente ao uso da GPU, uma vez que alterações no desenho algorítmico ou na parametrização podem, por si só, explicar diferenças observadas. Neste trabalho, os parâmetros

e operadores são mantidos fixos entre as variantes, e as diferenças de desempenho observadas são discutidas à luz do protocolo experimental apresentado no [Capítulo 3].

Neste trabalho, busquei implementar as variantes do algoritmo sob as mesmas condições. A estrutura dos algoritmos é projetada para ser idêntica e otimizada tanto para CPU quanto para GPU, de forma justa, para todos os Algoritmos Genéticos e para o algoritmo de busca local 2-opt, variando apenas a forma como cada “módulo” estrutural do algoritmo é executado (em CPU ou em GPU) e o grau de paralelismo utilizado. A pergunta central pode ser formulada da seguinte forma:

Como comparar, de forma fiel, o impacto de diferentes estratégias de paralelização utilizando GPU no desempenho de um algoritmo memético (GA+2-opt) para o Problema do Caixeiro Viajante?

Responder a essa pergunta exige, ao mesmo tempo, um desenho experimental cuidadoso de hiperparâmetros e validade estatística (número significativo de instâncias, número adequado de repetições, métricas de qualidade e tempo). Os parâmetros básicos do GA — como tamanho de população n_{pop} , taxa de mutação p_{mut} , tamanho do torneio k_{trnmt} , número de iterações de 2-opt por indivíduo n_{2-opt} e número de repetições por combinação algoritmo–instância n_{reps} — são escolhidos com base em recomendações da literatura de algoritmos evolutivos (16,12) e em análises específicas documentadas nos relatórios técnicos do projeto. Esses valores e sua motivação são apresentados em detalhe no Capítulo 3.

1.3 Objetivo geral

O objetivo geral deste trabalho é investigar e quantificar o impacto de diferentes estratégias de paralelização em processadores gráficos, com foco em:

- **Tempo de execução:** medir e comparar o tempo requerido para cada variante do algoritmo memético GA+2-opt;
- **Qualidade das soluções:** avaliar o custo final das rotas obtidas e o *gap* relativo ao ótimo conhecido, validando a viabilidade da solução;
- **Manutenção de equivalência funcional:** garantir que as quatro variantes (GA-CPU, GA-Híbrido-Ingênuo, GA-Híbrido-Otimizado, GA-FullGPU) implementem exatamente a mesma lógica algorítmica, diferindo apenas na plataforma de execução e no grau de paralelismo;
- **Metodologia estatística padronizada:** aplicar testes de hipótese apropriados (testes de normalidade, pareados e rankings) conforme recomendações da literatura proposta por Demšar (17);
- **Alinhamento com taxonomia de paralelização:** situar as estratégias adotadas no contexto da taxonomia proposta por Crainic e Toulouse (4,18).

1.4 Objetivos específicos

Para viabilizar o objetivo geral, definem-se os seguintes objetivos específicos:

1. ****Implementar** quatro variações “isoalgorítmicas” [**isoalgorithm-disambiguation**] de um Algoritmo Genético híbrido com 2-opt para o TSP, que diferem apenas na forma e no local de execução (CPU ou GPU): [**isoalgorithm-disambiguation**]: O termo isoalgorítmico será utilizado neste trabalho para refletir variantes estruturalmente similares em sua composição.
 - uma versão executada puramente na CPU (**GA-CPU**), em que todas as operações (seleção, cruzamento, mutação, 2-opt e cálculo de custo) são executadas com NumPy, servindo como base em CPU para as comparações. Na prática, essa versão é utilizada como base principalmente para instâncias de menor porte (por exemplo, com $n_{coords} \leq 100$); para instâncias maiores, a comparação de tempo passa a considerar como referência a versão híbrida, conforme discutido no Capítulo 3.
 - uma versão híbrida com 2-opt executada na GPU e avaliação em CPU (**GA-Híbrido-Ingênuo** [**ingenuity-disambiguation**]), que inicia kernels **individualmente** para cada rota e transfere as soluções (ou circuitos) completos entre CPU e GPU a cada iteração;
 - uma versão híbrida otimizada (**GA-Híbrido-Otimizado**), em que 2-opt e o cálculo de custos são executados em GPU em lote (*batch*), reduzindo o número de lançamentos de kernel e o volume de dados transferidos;
 - uma versão totalmente em GPU (**GA-FullGPU**), na qual população, operadores genéticos, busca local e avaliação permanecem residentes na GPU durante toda a evolução.

Os nomes utilizados aqui seguem as implementações `GeneticAlgorithmCPU`, `GeneticAlgorithmHybridNaive`, `GeneticAlgorithmHybridOptimized` e `GeneticAlgorithmFullGPU` do framework experimental, cujos detalhes arquiteturais são apresentados no Capítulo 3. [**ingenuity-disambiguation**]: Na literatura, o termo “naive” (ingênuo) é frequentemente utilizado para descrever implementações simples que não exploram otimizações avançadas.

2. **Selecionar um conjunto de instâncias da TSPLIB** com diferentes faixas de tamanho (pequenas $n_{coords} \leq 100$, médias $100 < n_{coords} \leq 400$ e grandes $n_{coords} > 400$). Essa seleção visa demonstrar os ganhos de desempenho que podem ser atingidos e garantir diversidade suficiente para analisar ganhos em escala. A divisão de faixas adotada é inspirada em estudos prévios com a TSPLIB (10), mas levemente ajustada para refletir o foco deste trabalho nas instâncias pequenas, médias e grandes selecionadas. O ajuste consiste em adotar explicitamente o número de coordenadas $n_{coords} = 100$ e $n_{coords} = 400$ como limites entre as faixas, em linha com o

limiar prático utilizado para a comparação CPU/GPU e com a distribuição de tamanhos das instâncias efetivamente utilizadas nos experimentos.

3. **Definir um protocolo experimental reprodutível**, incluindo número de repetições por instância e algoritmo, critérios de parada, parâmetros do GA e limites práticos impostos pela capacidade de memória da GPU. O desenho desse protocolo segue recomendações da literatura de comparação de algoritmos e de testes estatísticos (17) e é detalhado no Capítulo 3.
4. **Aplicar um conjunto de testes estatísticos apropriados** para comparação de algoritmos, incluindo testes de normalidade (Shapiro–Wilk), testes pareados paramétricos (teste *t-pareado*) e não paramétricos (Wilcoxon), análise de rankings em múltiplas instâncias (teste de Friedman com pós-teste de Nemenyi) e medidas de tamanho de efeito (Cohen *d*), conforme recomendações em (17) e literatura correlata de teste de hipóteses. A metodologia estatística completa é apresentada no Capítulo 3.
5. **Analisar os resultados obtidos**, discutindo as possíveis condições em que cada variante algorítmica é vantajosa — como por exemplo, tamanho da instância n_{coords} , relação entre custos de comunicação e custo computacional, acesso à memória e volume de dados transferidos entre CPU e GPU nas direções host-to-device (H2D ou CPU $\rightarrow$ GPU) e device-to-host (D2H GPU $\rightarrow$ CPU) (2) —, limites práticos do uso de GPUs para o contexto de recursos limitados (paralelismo em uma placa gráfica já defasada e limitada) e as implicações para o projeto de algoritmos de roteamento em cenários reais, seguindo as taxonomias de paralelização de meta-heurísticas propostas por Crainic & Toulouse (4,19) e de estudos de caso em problemas de roteamento utilizando GPUs (5,6,7).

1.5 Justificativa

Do ponto de vista científico, o TSP continua sendo um problema de referência para avaliar novas ideias em heurísticas e meta-heurísticas (1). A vasta disponibilidade de estudos, instâncias utilizadas em larga escala e muitas soluções ótimas conhecidas, em particular as fornecidas pela TSPLIB (10), na qual o projeto se baseia, permite medir o desempenho de diferentes algoritmos tanto em termos de qualidade quanto em tempo de execução. Ao longo deste trabalho, será utilizada a notação n_{coords} para o número de coordenadas (cidades) de cada instância, n_{pop} para o tamanho da população do GA, n_{reps} para o número de repetições por combinação algoritmo–instância, T para tempos de execução médios e B_{H2D} , B_{D2H} para os volumes totais de dados transferidos entre CPU e GPU nas direções host-to-device e device-to-host, respectivamente. Os detalhes de como esses elementos se relacionam com os limites de memória de cada variante são discutidos no Capítulo 3.

No campo da computação de alto desempenho, diferentes trabalhos relatam ganhos relevantes ao portar heurísticas de melhoria e algoritmos evolutivos para GPU, inclusive em variantes do TSP (5,6,7). Esses resultados indicam que GPUs

são um hardware promissor para esse tipo de algoritmo, mas também evidenciam um problema recorrente: em muitas comparações, as versões em CPU e GPU diferem não apenas na plataforma de execução, mas também em detalhes de implementação e parametrização, o que dificulta isolar o efeito específico da paralelização.

Além disso, para que conclusões sobre desempenho sejam confiáveis, não basta observar poucos experimentos isolados. É necessário adotar um desenho experimental com múltiplas repetições, tanto para instâncias quanto para, medidas de dispersão e testes de hipótese adequados, como discutido na literatura de comparação de algoritmos (17). Neste trabalho, esses cuidados são incorporados ao experimento, de modo que as diferenças observadas entre variantes em CPU e GPU possam ser atribuídas, com maior segurança, às estratégias de paralelização avaliadas.

Este trabalho justifica-se por combinar três elementos que, em conjunto, fortalecem a contribuição científica:

1. **Implementação de heurísticas clássicas em GPU** (particularmente 2-opt e operadores do AG), explorando o paralelismo de forma explícita.
2. **Comparação entre variantes funcionalmente equivalentes do algoritmo em CPU e GPU**, mantendo fixos operadores, parâmetros e critérios de parada, de modo a isolar o efeito da plataforma de execução. [^small-variations] [^small-variations]: Algumas variações dentro deste contexto são esperadas, mas de modo geral, o algoritmo segue a mesma estrutura. Sempre há a possibilidade de haver pequenas variações, mas o autor busca manter, ao máximo, a consistência entre as variantes.
3. **Aplicação de metodologia estatística rigorosa**, com múltiplas repetições por instância, testes de hipótese adequados e medidas de tamanho de efeito.

Os capítulos seguintes discutem em que condições cada variante é vantajosa, quais os limites práticos do uso de GPUs e recursos limitados e como os resultados dialogam com estudos prévios de heurísticas em GPU.

1.6 Organização do trabalho

Este texto está organizado da seguinte forma:

- **Capítulo 2 – Fundamentação Teórica / Revisão Bibliográfica:** apresenta os conceitos básicos de otimização combinatória e do TSP, revisa heurísticas de construção e melhoria (com ênfase em 2-opt), discute algoritmos genéticos e meta-heurísticas híbridas, introduz princípios de computação em GPU e paralelização de meta-heurísticas, e resume abordagens estatísticas para comparação de algoritmos.
- **Capítulo 3 – Materiais e Métodos:** descreve o ambiente computacional, a arquitetura do framework desenvolvido (incluindo abstrações de backend e estratégias isoalgorítmicas), os detalhes do Algoritmo Genético híbrido

com 2-opt nas quatro variantes consideradas, o conjunto de instâncias TSPLIB utilizado, os parâmetros experimentais e o protocolo estatístico adotado.

- **Capítulo 4 – Resultados:** apresenta os resultados numéricos dos experimentos, incluindo estatísticas por instância, análises agregadas por faixa de tamanho, medidas de ganho de performance e qualidade das soluções, além da interpretação dos testes estatísticos aplicados.
- **Capítulo 5 – Discussão:** interpreta criticamente os resultados à luz da literatura, discute as implicações dos achados para o projeto de algoritmos de roteamento em GPU e analisa limitações do estudo.
- **Capítulo 6 – Conclusões e Trabalhos Futuros:** sintetiza as principais contribuições do trabalho, responde explicitamente aos objetivos propostos e indica possíveis extensões, como a aplicação do framework a problemas de roteirização mais complexos.
- Elementos pós-textuais, como referências, glossário, apêndices técnicos e anexos com tabelas completas de resultados, são apresentados ao final do documento, conforme normas da instituição.

Capítulo 2 - Revisão bibliográfica

2. Fundamentação teórica e revisão bibliográfica

Este capítulo apresenta conceitos teóricos e a bibliografia utilizada para contextualizar o problema estudado e as escolhas metodológicas adotadas. São discutidos o Problema do Caixeiro Viajante (TSP) no contexto da otimização combinatória (1,20), sua importância geral e exemplos de utilização. Também são apresentadas generalizações do problema e sua relação com problemas de roteamento de veículos e variantes correlatas (21,22,23), heurísticas e meta-heurísticas clássicas para o TSP (13,14,24), aprofundando-se então no tópico de algoritmos genéticos e algoritmos meméticos (11,12), princípios de computação em GPU (2,5,6), taxonomias de paralelização de meta-heurísticas (4,18,19) e de comparação estatística de algoritmos de otimização (17).

2.1 Otimização combinatória e o Problema do Caixeiro Viajante

O TSP pode ser formulado, na versão simétrica¹, como um problema de encontrar um ciclo hamiltoniano de custo mínimo — podendo este ser a distância percorrida, combustível gasto, ou mesmo uma combinação de fatores. Para fins deste trabalho, o custo será também chamado de distância — em um grafo completo não direcionado $G = (V, E)$, no qual cada vértice em V representa uma cidade e cada aresta $(i, j) \in E$ possui um custo $c_{ij} \geq 0$ entre as cidades i e j (1). O

¹Na versão assimétrica do TSP, os custos de viagem entre pares de cidades podem diferir dependendo da direção (ou seja, $c_{ij} \neq c_{ji}$). Em TSPs simétricos, o custo do cálculo de distâncias pode ser representado por uma matriz triangular que armazena apenas uma das metades (superior ou inferior) sem a diagonal, o que reduz o número de entradas únicas de n^2 para $n(n-1)/2$.

objetivo é determinar uma combinação das cidades que minimize a soma total dos custos, retornando à cidade de origem. Geralmente em aplicações práticas assume-se que a matriz de custos é métrica e euclidiana, isto é, os custos derivam de distâncias euclidianas entre coordenadas no plano.

Do ponto de vista da complexidade, o TSP é um problema NP-difícil em sua forma de otimização e NP-completo em sua forma de decisão (1). Isso significa que, em geral, não se conhece um algoritmo em tempo polinomial que resolva instâncias arbitrárias em grande escala. Mesmo que seja relativamente fácil verificar o custo de uma ou várias soluções candidatas, não podemos afirmar que esta é a solução exata em tempo polinomial. Algoritmos modernos como **Held-Karp** (25) com uma complexidade temporal $O(n^2 2^n)$ e o algoritmo **Concorde** (1) com uma complexidade temporal indefinida, conseguem resolver instâncias com milhares de cidades, mas seu tempo de execução cresce exponencialmente ($O(n^2 2^n)$), o que, embora assintoticamente muito superior à força bruta ($O(n!)$), ainda o torna impraticável para instâncias muito grandes ².

Ao longo das últimas décadas, o TSP consolidou-se também como um padrão de avaliação empírica de algoritmos, graças à disponibilidade de coleções de instâncias padronizadas, como a TSPLIB (10). Essas coleções incluem instâncias com diferentes tamanhos, estruturas e origens (geográficas, sintéticas, industriais), a maioria delas com soluções ótimas conhecidas e obtidas por métodos exatos. No presente trabalho, são utilizadas algumas dessas instâncias como base para avaliar e comparar heurísticas e variantes paralelas de um algoritmo memético (combinações de algoritmos evolutivos com busca local) (11,7).

2.2 Heurísticas para o TSP

Devido à dificuldade de resolver instâncias grandes do TSP exatamente, uma vasta literatura de heurísticas tem sido desenvolvida para produzir boas soluções em tempos de computação aceitáveis (1). De forma geral, heurísticas podem ser agrupadas em dois grandes tipos: heurísticas de construção e heurísticas de melhoria.

Heurísticas de construção produzem uma solução viável “do zero”, frequentemente seguindo regras simples, também conhecidas como algoritmos gulosos, como escolher iterativamente o vizinho mais próximo ou inserir cidades em posições que causem o menor aumento de custo. Exemplos incluem a heurística do vizinho mais próximo (nearest neighbor — NN), heurísticas de inserção (insertion heuristics) e variantes baseadas em árvores de extensão mínimas (Minimum Spanning Trees — MSTs), como o Algoritmo de Christofides. Embora rápidas e fáceis de implementar, essas estratégias tendem a gerar soluções de qualidade moderada, servindo sobretudo como ponto de partida para métodos

²O algoritmo Held-Karp “troca” a complexidade temporal da força bruta $O(n!)$ por uma complexidade espacial $O(n 2^n)$, tornando sua viabilidade limitada a instâncias com $n_{coords} \approx 40$ cidades. Já o algoritmo Concorde, baseado em técnicas de ramificação e corte, é capaz de resolver instâncias com até 85.900 cidades, mas seu desempenho depende fortemente da estrutura específica da instância (1).

mais sofisticados. Resultados clássicos da literatura mostram que, em instâncias métricas, heurísticas baseadas em MST e em Christofides podem oferecer garantias de aproximação sobre o custo ótimo de $\frac{3}{2}B^*$, aspecto discutido em textos de referência em logística e roteamento (26).

Heurísticas de melhoria partem de uma solução inicial e aplicam sucessivos movimentos locais que procuram reduzir seu custo. Entre essas, as chamadas k -opt são particularmente influentes: um movimento 2-opt consiste em remover duas arestas de um circuito e reconectar os segmentos resultantes, escolhendo a reconexão que produz um circuito ainda viável e de menor comprimento (13). Em instâncias euclidianas é comum interpretar o 2-opt como um mecanismo para eliminar cruzamentos, mas, genericamente, ele também pode melhorar circuitos que não apresentam interseções evidentes, ao substituir pares de arestas por combinações de menor custo (13). Movimentos 3-opt e extensões mais complexas, como o algoritmo de Lin–Kernighan, generalizam essa ideia (14). Estas e outras heurísticas de melhoria desempenham papel central em muitos algoritmos modernos para TSP, tanto como procedimentos isolados quanto como componentes de meta-heurísticas, sendo utilizadas como procedimentos de busca local.

Neste trabalho, a heurística 2-opt é utilizada como busca local básica acoplada ao Algoritmo Genético, em linha com estudos que combinam heurísticas de construção simples com procedimentos de melhoria mais complexos para obter soluções de alta qualidade em tempo razoável (24,11,7).

2.2.1 Entendendo o algoritmo 2-opt A formulação básica de um movimento 2-opt pode ser descrita da seguinte forma (13): dado um circuito $C = (v_0, v_1, \dots, v_{n-1}, v_0)$ e dois índices i e j tais que $0 \leq i < j < n - 1$, considera-se a remoção das arestas (v_i, v_{i+1}) e (v_j, v_{j+1}) e a inserção das arestas (v_i, v_j) e (v_{i+1}, v_{j+1}) . A variação de custo associada ao movimento é dada por

$$\Delta C = (c_{i,j} + c_{i+1,j+1}) - (c_{i,i+1} + c_{j,j+1}),$$

onde $c_{i,j}$ denota o custo (ou distância) entre as cidades i e j . Um movimento 2-opt é aceitável quando $\Delta C < 0$, isto é, quando a substituição das arestas reduz o comprimento total do circuito (13).

Em implementações práticas, percorrem-se pares de índices (i, j) em alguma ordem predefinida até que nenhum movimento que gere um custo menor seja encontrado ou até que um **limite máximo de iterações** seja atingido. O conceito de “iteração” no contexto de 2-opt refere-se a uma passagem completa sobre a vizinhança do circuito: em cada iteração, examina-se um conjunto de pares (i, j) e aplicam-se todos os movimentos que reduzem o custo.

Quando determinamos “ $\kappa = 10$ iterações em 2-opt”, significa que o algoritmo realizará até 10 dessas passagens completas, parando antes se não houver mais melhorias possíveis. Sem um limite máximo, o algoritmo continuaria até atingir

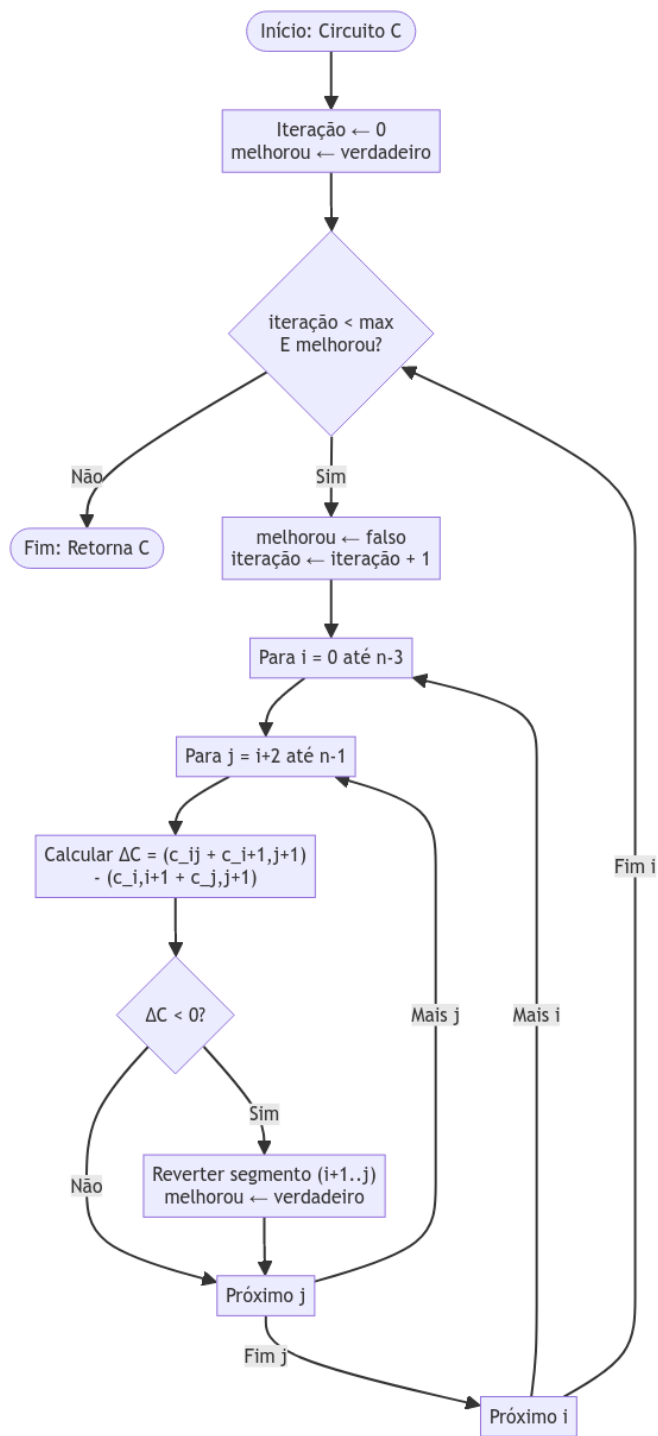
um ótimo local — o que pode ser custoso computacionalmente em circuitos grandes (14). No contexto de algoritmos meméticos (Seção 2.5), o número de iterações de 2-opt por indivíduo é um parâmetro crucial que equilibra intensidade da busca local com custo computacional: muitas iterações podem refinar excessivamente cada solução — o alto custo computacional despendido segue a lei dos rendimentos decrescentes —, enquanto poucas podem deixar melhorias óbvias sem explorar.

De forma mais detalhada, o algoritmo 2-opt clássico pode ser descrito como um procedimento iterativo de busca local aplicada a um único circuito. Em linhas gerais, o algoritmo segue os passos:

1. Começar com um circuito viável inicial C (obtido por uma heurística de construção qualquer).
2. Definir uma ordem de varredura para pares de índices (i, j) com $0 \leq i < j < n - 1$. No presente trabalho, utiliza-se a ordem lexicográfica padrão, em que para cada i , varia-se $j = i + 2$ até $n - 1$ (a restrição $j \geq i + 2$ evita movimentos triviais [trivial] e garante que o segmento a ser revertido tenha comprimento mínimo (13)). [trivial]: Movimentos triviais são aqueles que não alteram efetivamente o circuito, como tentar reverter um segmento de i a $i + 1$, pois $c_{i,i+1} = c_{i+1,i}$ é uma aresta única e sua reversão não muda o circuito, no caso do TSP clássico explorado neste documento. O mesmo não se aplica para o ATSP.
3. Para cada par (i, j) , calcular ΔC conforme a expressão acima.
4. Se $\Delta C < 0$, aplicar o movimento 2-opt correspondente, o que equivale a reverter o segmento $(v_{i+1}, \dots, v_j)$ do circuito, obtendo um novo circuito C' , e marcar que houve melhora.
5. Repetir o processo de varredura enquanto forem encontrados movimentos com $\Delta C < 0$ (isto é, até atingir um ótimo local em relação à vizinhança 2-opt) ou até que um número máximo de iterações seja alcançado.

Em uma implementação ingênua, a vizinhança 2-opt de um circuito com n cidades possui ordem $O(n^2)$ movimentos possíveis, o que implica um custo potencialmente elevado quando todos os pares (i, j) são examinados de forma exaustiva. Diversas otimizações são discutidas na literatura, como o uso de listas de vizinhança, estruturas de dados para poda de movimentos claramente não promissores e estratégias de parada antecipada (14,11). No contexto desta monografia, o foco recai principalmente na forma como esse procedimento é paralelizado e acoplado ao Algoritmo Genético, mais do que em otimizações finas da vizinhança em CPU.

A Figura~1 ilustra a lógica do algoritmo 2-opt clássico: o laço externo continua enquanto houver melhorias e o número de iterações não exceder o limite. Em cada iteração, todos os pares (i, j) são examinados; movimentos com $\Delta C < 0$ são aplicados imediatamente, revertendo o segmento do circuito.



Figura~1: Fluxograma do algoritmo 2-opt.

Input: Initial tour C, max_iterations

Output: Optimized tour C

```
iteration = 0
improved = true

while improved and iteration < max_iterations do
    improved = false
    iteration = iteration + 1

    for i = 0 to n-3 do
        for j = i+2 to n-1 do
            delta = dist(i, j) + dist(i+1, j+1) - (dist(i, i+1) + dist(j, j+1))

            if delta < 0 then
                Reverse segment C[i+1...j]
                improved = true
            end if
        end for
    end for
end while

return C
```

2.3 Algoritmos genéticos

Algoritmos Genéticos (Genetic Algorithms – GAs) são meta-heurísticas inspiradas em princípios de evolução biológica, nas quais uma população de soluções candidatas é iterativamente modificada por operadores análogos à seleção natural, recombinação e mutação (12,16). Na sua forma mais simples, um AG mantém uma população de indivíduos representando soluções para o problema; em cada geração, indivíduos são selecionados com base em uma função de aptidão (fitness), recombinados por operadores de cruzamento e perturbados por operadores de mutação. A nova população resultante substitui total ou parcialmente a anterior, e o processo se repete até que um critério de parada seja satisfeito.

No contexto do TSP, é comum representar cada solução como uma permutação das cidades, utilizar operadores de cruzamento especializados para rotas — como Order Crossover (OX), Partially Mapped Crossover (PMX), entre outros — e empregar mutações que preservem a viabilidade da permutação, como trocas de posição entre duas cidades (11). Estudos clássicos mostram que GAs podem produzir soluções competitivas para o TSP quando combinados com operadores e parâmetros adequados (12).

Do ponto de vista formal, um GA opera sobre uma população $P(t) =$

$\{x_1^{(t)}, x_2^{(t)}, \dots, x_{n_{pop}}^{(t)}\}$ de indivíduos (soluções candidatas) na geração t . Cada indivíduo x_i possui um valor de aptidão (*fitness*) $f(x_i)$ que mede a qualidade da solução; no caso do TSP, $f(x_i)$ corresponde ao custo do circuito representado por x_i , e busca-se minimizar esse valor. A cada geração, o GA aplica três operadores principais (12,16):

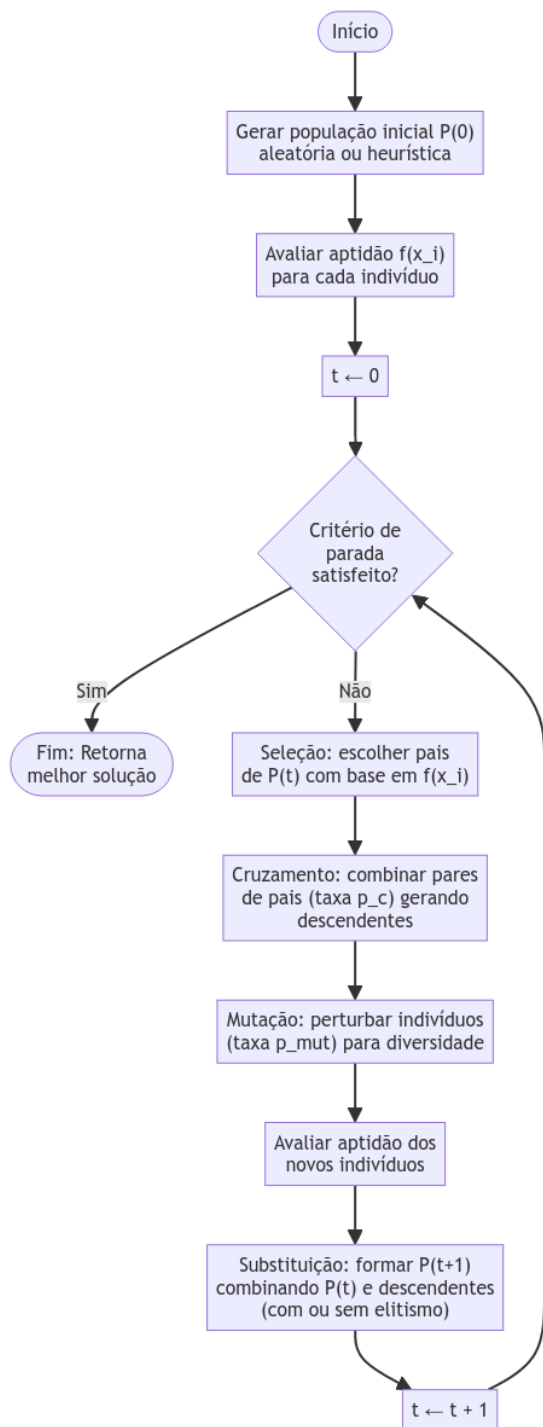
1. **Seleção:** escolhe indivíduos de $P(t)$ com probabilidade proporcional (ou baseada em ranking/torneio) à sua aptidão, favorecendo soluções de melhor qualidade. Formalmente, a probabilidade de selecionar x_i pode ser dada por $p_i = f(x_i) / \sum_{j=1}^{n_{pop}} f(x_j)$ (seleção proporcional) ou por esquemas de torneio, nos quais um subconjunto aleatório de indivíduos compete e o melhor é escolhido.
2. **Cruzamento (recombinação):** combina pares de indivíduos selecionados (“pais”) para produzir descendentes. No TSP, operadores como OX e PMX preservam a viabilidade das permutações, herdando subsequências de um dos pais e preenchendo as posições restantes com a ordem do outro (11). A taxa de cruzamento p_c controla a fração de pares submetidos a esse operador.
3. **Mutação:** introduz pequenas perturbações aleatórias em indivíduos, mantendo diversidade na população. No TSP, mutações típicas incluem troca de duas cidades ou reversão de segmentos. A taxa de mutação p_{mut} controla a probabilidade de cada indivíduo sofrer mutação.

Após aplicar esses operadores, forma-se a nova população $P(t+1)$ por meio de um esquema de substituição (geracional, com elitismo, steady-state, etc.). A Figura~2 ilustra o fluxo geral de um GA (12,16): a população evolui iterativamente por meio de seleção (escolha de pais com base em aptidão), cruzamento (combinação de pares de pais gerando descendentes), e mutação (perturbações para diversidade), até que um critério de parada seja atingido.

```

BEGIN AGA
  Make initial population at random.
  WHILE NOT stop DO
    BEGIN
      Select parents from the population.
      Produce children from the selected parents.
      Mutate the individuals.
      Extend the population adding the children to it.
      Reduce the extended population.
    END
  Output the best individual found.
END AGA

```

Figura~2: Fluxograma de Algoritmo Genético.

Input: Population size N , mutation rate p_{mut} , crossover rate p_{cross}

Output: Best solution found

Initialize population $P(0)$ randomly

Evaluate fitness of all individuals in $P(0)$

$t = 0$

while not stop_condition do

$P_{new} = \{\}$

 // Elitism: keep best individuals

 Add best of $P(t)$ to P_{new}

 while size(P_{new}) < N do

 // Selection

 parent1 = TournamentSelection($P(t)$)

 parent2 = TournamentSelection($P(t)$)

 // Crossover

 if random() < p_{cross} then

 child = OrderCrossover(parent1, parent2)

 else

 child = parent1

 end if

 // Mutation

 if random() < p_{mut} then

 SwapMutation(child)

 end if

 Add child to P_{new}

 end while

$P(t+1) = P_{new}$

 Evaluate fitness of $P(t+1)$

$t = t + 1$

end while

return Best of $P(t)$

2.4 Heurísticas híbridas

De forma ampla, heurísticas híbridas combinam duas ou mais estratégias de busca, procurando explorar sinergias entre métodos com forças complementares.

No contexto de problemas de roteamento e, em particular, do TSP, é comum combinar heurísticas de construção (como vizinho mais próximo, inserções ou heurísticas baseadas em MST) com heurísticas de melhoria (como movimentos k -opt) ou com meta-heurísticas populacionais (como GAs), de modo que uma componente forneça boas soluções iniciais e outra detalhe a exploração de vizinhanças (24).

Diversos autores classificam heurísticas híbridas em categorias como: (i) **hibridização em nível de solução**, na qual uma meta-heurística utiliza sistematicamente uma busca local (ou outro procedimento) para melhorar indivíduos candidatos; (ii) **hibridização em nível de algoritmo**, em que diferentes técnicas (por exemplo, um método exato e um heurístico) são combinadas em fases distintas ou em esquemas de cooperação; e (iii) **hibridizações multiestágio**, nas quais diferentes heurísticas atuam em momentos distintos do processo de resolução (inicialização, intensificação, diversificação, etc.) (11,24). Essas categorias ajudam a organizar o espectro de abordagens híbridas reportadas na literatura, embora nem sempre haja consenso absoluto sobre as fronteiras entre elas.

Uma extensão importante dessa ideia é o conceito de algoritmos meméticos, nos quais operadores evolutivos (seleção, cruzamento, mutação) são combinados com heurísticas de busca local aplicadas a indivíduos da população, de forma sistemática (11). Em problemas de roteamento, isso resulta em esquemas GA+LS, nos quais um GA guia a exploração global do espaço de soluções e um procedimento de melhoria, como 2-opt ou Lin-Kernighan, refina soluções promissoras. Esses algoritmos tendem a oferecer melhor equilíbrio entre exploração e intensificação do que GAs “puros”, ao custo de maior tempo computacional por iteração.

Trabalhos como o de Fujimoto e Tsutsui (7) exploram justamente essa combinação GA+busca local para o TSP em ambientes de computação paralela, motivando a adoção de uma abordagem semelhante neste estudo. A próxima subseção aprofunda o conceito de algoritmos meméticos e a subseção 2.5.1 descreve, em linhas gerais, o esquema GA+2-opt adotado nesta monografia.

2.5 Algoritmos meméticos

Algoritmos meméticos podem ser vistos como uma família de meta-heurísticas evolutivas que incorporam, explicitamente, operadores de busca local ao ciclo de evolução populacional (11). A metáfora usual associa o termo “meme” a unidades de informação que se propagam e se transformam, de modo análogo a genes, mas em um nível mais alto de organização: além da recombinação e mutação de indivíduos, há um processo de “aprendizado” local que modifica soluções de forma dirigida.

Em um algoritmo memético típico, a cada geração, parte dos indivíduos (por exemplo, os mais aptos ou uma amostra da população) é submetida a uma busca local, que pode ser uma heurística de melhoria como 2-opt, 3-opt ou Lin-Kernighan no caso do TSP. Esse procedimento permite refinar soluções já

boas, acelerando a convergência em direção a ótimos locais de alta qualidade. O GA fornece a diversidade global por meio de operadores de cruzamento e mutação, enquanto a busca local intensifica a exploração em torno de regiões promissoras do espaço de soluções.

O desenho de um algoritmo memético envolve decisões importantes, como: (i) **quais indivíduos** serão submetidos à busca local (apenas a elite, um subconjunto aleatório, toda a população); (ii) **com que frequência** a busca local será aplicada (toda geração, gerações alternadas, fases específicas do processo); e (iii) **com que intensidade** cada chamada de busca local será executada (por exemplo, número máximo de iterações de 2-opt). Essas escolhas impactam o balanço entre qualidade das soluções e custo computacional, bem como a diversidade mantida na população.

Estudos fundamentais, como o de Larrañaga et al., indicam que os algoritmos meméticos são extremamente eficientes para o Problema do Caixeiro Viajante (TSP). Estes métodos funcionam melhor quando combinam o cruzamento de rotas com técnicas de melhoria intensiva. Neste trabalho, utilizamos um **Algoritmo Genético adaptado para rotas**, que possui as seguintes características:

1. **Representação Numérica:** Cada solução é um circuito C denotando uma sequência de números inteiros representando a ordem das cidades.
2. **Seleção por Torneio:** Os “pais” são escolhidos através de competições diretas entre pequenos grupos de indivíduos.
3. **Cruzamento Ordenado (OX):** Um operador que preserva a ordem relativa das cidades herdadas dos pais.
4. **Mutação por Troca:** Pequenas alterações feitas trocando duas cidades de posição na rota.
5. **Substituição com Elitismo:** A nova geração substitui a antiga, mas preserva obrigatoriamente as melhores soluções encontradas até o momento.
6. **Hibridização:** Após essas etapas, aplica-se uma rotina de busca local (2-opt) para refinar os resultados.

A viabilidade de acelerar esse processo usando placas de vídeo (GPU) foi comprovada por Fujimoto e Tsutsui. Eles implementaram este algoritmo em uma GPU (modelo GTX285) utilizando técnicas eficientes de soma paralela para gerenciar os dados. Ao comparar o desempenho contra um processador tradicional operando com **apenas um núcleo** (*single-core*), observaram que a GPU foi até **24,2 vezes mais rápida** na instância **ts225**. Embora a comparação tenha sido feita contra um processador mais antigo, o resultado prova que problemas de tamanho médio (200-300 cidades) conseguem tirar proveito do paralelismo massivo da GPU.

O processo formal de refinamento segue a lógica abaixo:

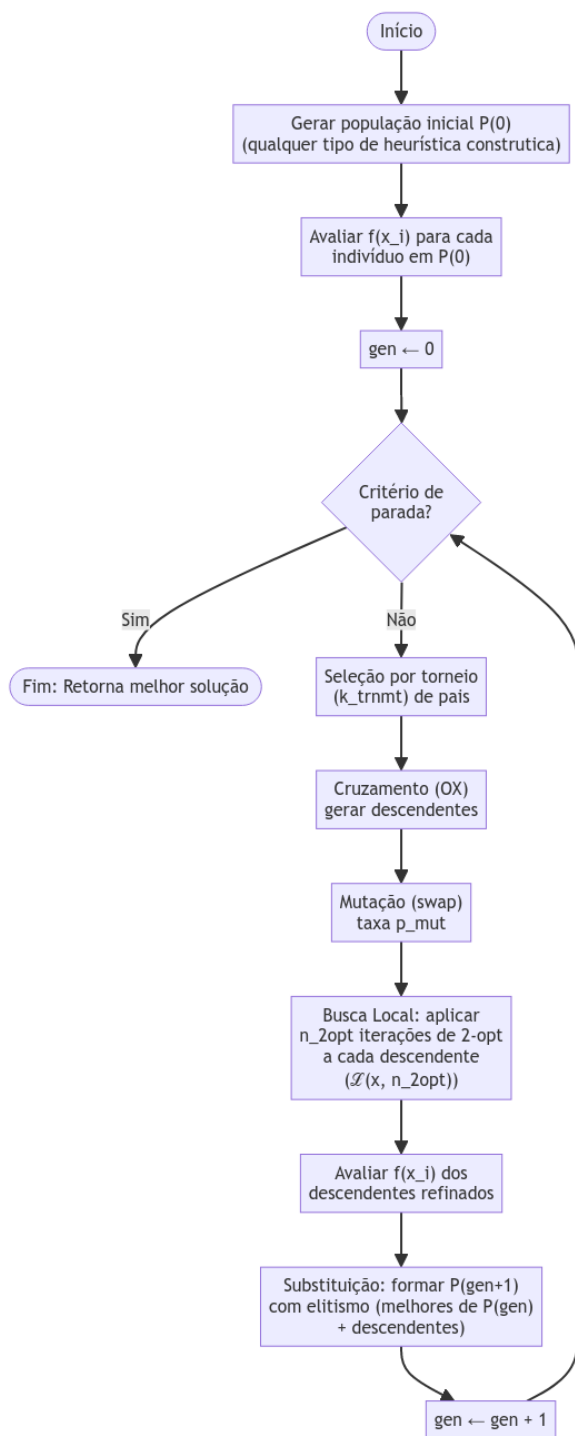
1. Considere uma população inicial $P(t)$.
2. Aplique os operadores genéticos (cruzamento e mutação) para criar descendentes.
3. Execute a função de **Busca Local Truncada**, denotada por $\mathcal{L}(x, \kappa)$.

4. Esta função aplica melhorias na rota (2-opt) repetidamente até que:
 - A rota não possa mais ser melhorada (ótimo local); **OU**
 - O número máximo de varreduras (κ) seja atingido.
5. O parâmetro κ serve como um “freio”: ele limita o tempo gasto refinando cada solução para evitar que o algoritmo fique lento demais, garantindo um equilíbrio entre qualidade e velocidade, mantendo a eficiência do algoritmo.

2.5.1 Algoritmo Genético + 2-opt O esquema GA+2-opt adotado neste trabalho segue a linha de algoritmos meméticos padrões para o TSP (11), nos quais um Algoritmo Genético opera sobre uma população de permutações de cidades e, em momentos específicos do ciclo evolutivo, aplica-se uma rotina de busca local 2-opt a alguns indivíduos. Em alto nível, cada iteração (geração) do algoritmo pode ser descrita pelos seguintes passos:

1. **Inicialização** ($t = 0$): gerar uma população inicial $P(0)$ de rotas, por exemplo, a partir de permutações aleatórias ou de heurísticas de construção simples (vizinho mais próximo, inserções).
2. **Avaliação** (*fitness*): calcular o custo (comprimento) $f(x_i)$ de cada rota $x_i \in P(t)$, que servirá como medida de aptidão.
3. **Seleção**: escolher indivíduos para reprodução com base em seus valores de aptidão, utilizando seleção por torneio de tamanho k_{trnmt} , favorecendo soluções de menor custo.
4. **Cruzamento**: combinar pares de indivíduos selecionados por meio de Order Crossover (OX), produzindo descendentes que preservam a estrutura de permutação e herdam subsequências de ambos os pais.
5. **Mutação**: aplicar perturbações leves às rotas (trocas de posição de cidades, com taxa p_{mut}) para manter diversidade genética.
6. **Busca local 2-opt**: para cada descendente (ou um subconjunto, conforme a estratégia), aplicar $\mathcal{L}(x, n_{2\text{-opt}})$ — isto é, executar até $n_{2\text{-opt}}$ iterações de 2-opt — de modo a refinar localmente as rotas resultantes. Esta etapa é o que transforma o GA em um algoritmo memético, introduzindo intensificação explícita da busca.
7. **Substituição**: formar a nova população $P(t + 1)$ combinando os melhores indivíduos de $P(t)$ (elitismo) com os descendentes refinados, garantindo que as melhores soluções sejam preservadas ao longo das gerações.

O processo repete-se até que um critério de parada seja satisfeito (número máximo de gerações, estagnação da aptidão ou atingimento do custo ótimo conhecido). A Figura~3 ilustra o fluxo completo do GA+2-opt memético (11): a busca local de 2-opt (etapa destacada) é aplicada após os operadores genéticos (seleção por torneio, cruzamento OX, mutação swap), refinando cada descendente com até $n_{2\text{-opt}}$ iterações de 2-opt ($\mathcal{L}(x, n_{2\text{-opt}})$) antes da formação da nova população. Esse acoplamento caracteriza o algoritmo como memético, combinando exploração global (GA) e intensificação local (2-opt).



Figura~3: Fluxograma do algoritmo GA+2-opt.

No contexto deste trabalho, essa etapa de busca local será posteriormente mapeada para diferentes implementações em CPU e GPU (GA-CPU, GA-Híbrido-Ingênuo, GA-Híbrido-Otimizado, GA-FullGPU), mantendo a mesma lógica de aplicação de 2-opt (mesma vizinhança, mesmos limites de iterações n_{2-opt}), de forma a preservar o caráter estruturalmente equivalente das variantes. Os detalhes operacionais, incluindo a forma precisa de parametrizar n_{2-opt} , n_{pop} , p_{mut} e os critérios de parada do GA, são apresentados no Capítulo 3.

2.6 Computação em GPU e paralelização de meta-heurísticas

Processadores gráficos (GPUs) evoluíram, nas últimas décadas, de dispositivos voltados principalmente para renderização gráfica para plataformas de computação de uso geral (GPGPU - General-purpose computing on GPUs), amplamente utilizadas em aplicações científicas e de inteligência artificial (2). O modelo de programação CUDA, por exemplo, organiza o trabalho em grades (*grids*) de blocos de threads, seguindo um paradigma de execução massivamente paralelo próximo ao SIMT (Single Instruction, Multiple Threads), relacionado às classificações de arquiteturas de Flynn (3). Nessa configuração, milhares de threads executam o mesmo kernel sobre dados distintos, o que é adequado a tarefas com alto grau de paralelismo em dados.

Internamente, as threads em uma GPU são agrupadas em *warps* (tipicamente 32 threads em arquiteturas NVIDIA), que executam a mesma instrução simultaneamente (SIMT - *Single Instruction, Multiple Threads*) (2). Isso difere do modelo SIMD (*Single Instruction, Multiple Data*) tradicional de CPUs, onde uma única instrução opera sobre vetores de dados, pois no SIMT cada thread possui seu próprio contador de programa e estado de registradores, permitindo maior flexibilidade, embora a divergência de fluxo dentro de um warp possa penalizar o desempenho. Segundo a taxonomia de Flynn (3), conforme introduzido na Seção 1.1, enquanto CPUs modernas operam predominantemente no modelo MIMD (*Multiple Instruction, Multiple Data*) com extensões SIMD ³, as GPUs maximizam o *throughput* dedicando transistores para processamento massivo de dados (SIMD/SIMT) em detrimento de caches grandes e controle de fluxo, sendo ideais para cargas de trabalho onde a mesma operação é aplicada a grandes volumes de dados independentes.

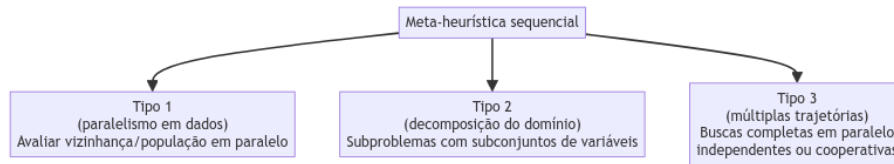
2.6.1 Taxonomia de paralelização segundo Crainic e Toulouse A taxonomia de Crainic e Toulouse (4,18) classifica as estratégias de paralelização de meta-heurísticas em três categorias principais, cada uma com adequação distinta a arquiteturas de hardware:

³Embora CPUs sejam classificadas como MIMD (Multiple Instruction, Multiple Data), processadores modernos incorporam extensões vetoriais (SIMD) como AVX e SSE para acelerar operações em dados contíguos. Isso cria um modelo híbrido onde múltiplos fluxos de execução independentes (threads) podem, individualmente, explorar paralelismo de dados em nível de instrução.

1. **Tipo 1: Paralelismo de Baixo Nível (Decomposição de Dados)**
 - **Descrição:** O foco é na paralelização de tarefas computacionalmente intensivas dentro de uma iteração da meta-heurística, como a avaliação da função objetivo ou a exploração de vizinhança. O fluxo de controle do algoritmo permanece centralizado.
 - **Adequação:** Ideal para arquiteturas **SIMD/SIMT (GPUs)**. Como a mesma operação (ex: cálculo de distância) é aplicada a muitos dados (ex: rotas, cálculos de matrizes), o *overhead* de inicializações de kernels e transferência é mínimo e o *throughput* é maximizado.
 - **Exemplo Real:** Rocki e Suda (6) utilizam paralelismo Tipo 1 para avaliar movimentos 2-opt em paralelo na GPU. Cada thread calcula o ganho de uma troca de arestas, permitindo explorar vizinhanças massivas em tempo reduzido.
2. **Tipo 2: Decomposição do Domínio**
 - **Descrição:** O espaço de busca ou as variáveis de decisão são particionados em subconjuntos (técnica comum em VRPs no geral), e a meta-heurística opera em paralelo sobre essas partições.
 - **Adequação:** Mais apropriado para arquiteturas **MIMD (Clusters ou CPUs Multicore)**. Subproblemas distintos podem ter tempos de convergência variados, o que causa divergência de execução em uma GPU (onde as 32 threads de cada warp devem ser sincronizadas).
 - **Exemplo Real:** Em problemas de Roteamento de Veículos com Múltiplos Depósitos (MDVRP), Diversos autores (26) descrevem estratégias onde cada “depósito” é otimizado por um processo independente em uma CPU, com trocas ocasionais de clientes entre depósitos.
3. **Tipo 3: Múltiplas Trajetórias (Buscas Independentes ou Cooperativas)**
 - **Descrição:** Múltiplas instâncias da meta-heurística (ou meta-heurísticas diferentes) exploram o espaço de soluções simultaneamente. Podem ser independentes (*multi-start*) ou cooperativas (troca de informações).
 - **Adequação:** Versátil. Funciona bem em:
 - **Clusters/CPUs** para algoritmos complexos e heterogêneos, através de técnicas de **multithreading** e **multiprocessing**, por exemplo.
 - **GPUs** para muitas execuções de algoritmos leves e idênticos, como *Island Models*.
 - **Combinados:** Processamento massivamente paralelo, conforme revisado por Schulz et al. (5).
 - **Exemplo Real:** Luong et al. (9) implementam um modelo de ilhas em GPU, onde milhares de pequenas populações evoluem em paralelo (cada uma em um bloco de threads), trocando periodicamente seus melhores indivíduos via memória compartilhada.

De forma resumida, a Figura~4 coloca lado a lado esses três tipos: a partir

de uma meta-heurística sequencial, pode-se paralelizar apenas a avaliação de vizinhança/população (Tipo 1), decompor o conjunto de variáveis em subproblemas (Tipo 2) ou rodar várias buscas completas em paralelo, independentes ou cooperativas (Tipo 3).



Figura~4: Resumo dos três tipos de paralelização de Crainic e Toulouse.

No contexto deste trabalho, as variantes desenvolvidas enquadram-se predominantemente no **Tipo 1**, focando na aceleração da avaliação de vizinhança 2-opt e operadores genéticos via paralelismo de dados massivo na GPU.

2.6.2 Aplicações de GPU a meta-heurísticas para o TSP Aplicações de GPU a meta-heurísticas para o TSP exploram principalmente paralelismo em dados, seja na avaliação em massa de rotas em algoritmos genéticos (por exemplo, via cálculos de redução em GPU), seja aplicando paralelismo a movimentos de vizinhança em heurísticas de melhoria (5,6,7). Outros trabalhos investigam paralelização de Recozimento Simulado, Colônias de Formigas (ACO) e outras meta-heurísticas em GPU, geralmente aproveitando o grande número de threads para explorar múltiplas soluções ou vizinhanças em cada passo da busca (27,28,29). Esses estudos motivam o uso de GPUs como plataforma para acelerar algoritmos meméticos, mas também evidenciam desafios relacionados à movimentação de dados entre CPU e GPU, à escolha de granularidade adequada de kernels, à ocupação dos multiprocessadores e às limitações de memória (VRAM). Esses aspectos são retomados no Capítulo 3, ao descrever o desenho dos kernels de 2-opt e as estratégias de controle de memória adotadas neste trabalho.

2.6.3 Contraste arquitetural: CPU vs GPU As figuras 5, 6 e 7 ajudam a visualizar, de forma simples, como CPUs e GPUs foram pensadas para resolver problemas diferentes. De maneira geral, CPUs têm poucos núcleos mais complexos e flexíveis. Uma parte grande da área do chip é usada para controle de fluxo, previsão de desvios e vários níveis de cache, o que favorece programas sequenciais, com muitas decisões e acesso irregular à memória. Nas GPUs acontece o contrário: a maior parte da área é ocupada por muitas unidades aritméticas simples (ALUs), organizadas em centenas ou milhares de núcleos em paralelo. Elas abrem mão de um controle sofisticado para ganhar vazão (*throughput*) quando muitas threads executam a mesma sequência de instruções sobre dados diferentes. Essa diferença é o motivo pelo qual GPUs funcionam bem para tarefas com muito paralelismo em dados, como avaliar muitas rotas de uma vez ou aplicar 2-opt em vários indivíduos em paralelo.

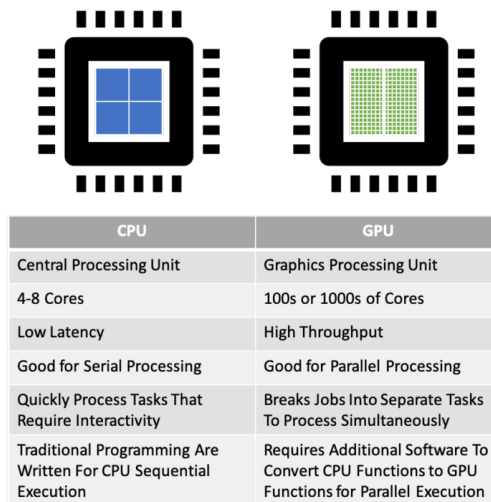


Figura 5: Arquitetura de CPU vs GPU. Fonte: LayerStack.

A Figura~6 mostra esse contraste na divisão da área do chip. Em uma CPU típica, poucos núcleos complexos dividem espaço com grandes caches e lógica de controle. Em uma GPU, o desenho é o inverso: a maior parte da área é dedicada a conjuntos de núcleos de processamento paralelos (por exemplo, *CUDA cores* em GPUs NVIDIA ou *stream processors* em GPUs AMD) e à memória de alta largura de banda. O controle é mais simples, mas o número de operações por segundo é muito maior quando o problema é bem mapeado para esse tipo de arquitetura (30).

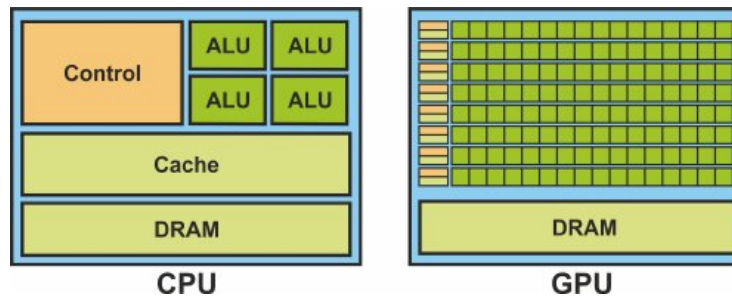


Figura 6: Alocação de silício em CPU e GPU. (30)

A Figura~7 resume como esses recursos aparecem na organização interna de uma GPU moderna. A memória global (DRAM) oferece grande largura de banda, mas latência alta; cada *streaming multiprocessor* (SM) possui memórias compartilhadas menores e registradores, usados pelas threads de um mesmo bloco. As threads são agrupadas em *warps* (tipicamente 32 threads nas GPUs NVIDIA)

que executam a mesma instrução ao mesmo tempo, seguindo o modelo SIMT (31). Quando as threads de um warp seguem caminhos de controle diferentes ou acessam a memória de forma muito irregular, parte dessa paralelização é perdida; quando o acesso é organizado e o fluxo de controle é parecido, o ganho de desempenho é grande.

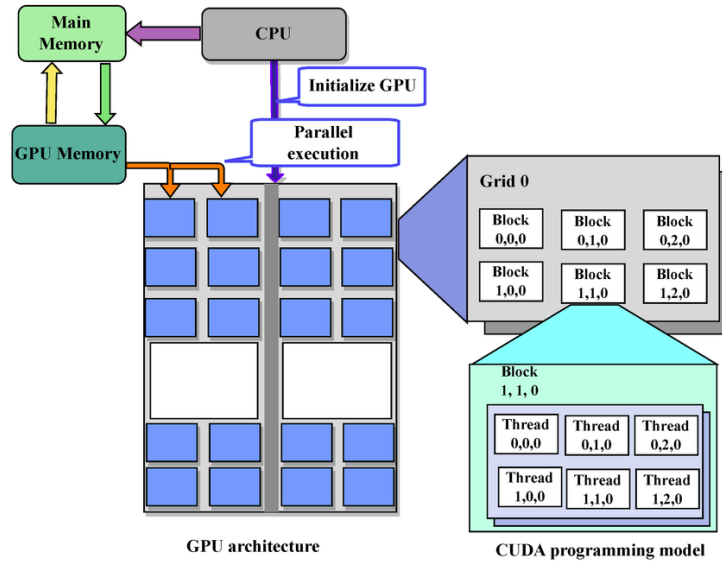


Figura 7: Organização interna de GPU moderna. (31)

Essas três figuras servem como pano de fundo para as decisões de projeto dos kernels CUDA apresentados no Capítulo 3: explorar paralelismo em dados (tipo 1) mapeando threads para indivíduos ou movimentos 2-opt, evitar ao máximo divergência dentro de um mesmo warp e organizar as leituras e gravações de memória global para aproveitar melhor a largura de banda disponível.

É fundamental distinguir, portanto, que enquanto CPUs modernas operam predominantemente no modelo MIMD (*Multiple Instruction, Multiple Data*) com extensões vetoriais SIMD (*Single Instruction, Multiple Data*) para acelerar operações específicas, as GPUs adotam o modelo SIMT (*Single Instruction, Multiple Threads*). No SIMD tradicional, uma única instrução controla múltiplos elementos de dados em *lockstep* estrito (como em instruções AVX). No SIMT, embora as threads de um mesmo *warp* compartilhem o contador de programa, cada thread possui seu próprio estado de registradores e pode, teoricamente, seguir fluxos distintos (divergência), embora isso acarrete penalidade de desempenho pela “serialização” dos caminhos divergentes, acarretando maior complexidade de sincronização entre cada *warp/bloco*. Essa flexibilidade do SIMT facilita a programação de algoritmos complexos como o 2-opt, onde a lógica de controle pode variar ligeiramente entre vizinhos, mas exige cuidado redobrado para manter a coerência de execução e maximizar a eficiência.

2.7 Comparação estatística de algoritmos de otimização

Meta-heurísticas estocásticas, como GAs e algoritmos meméticos, produzem resultados que variam de execução para execução devido ao uso de aleatoriedade em diversos pontos (inicialização, seleção, mutação, entre outros). Por esse motivo, a comparação de algoritmos de otimização não pode se basear em uma única execução por instância, tampouco apenas em médias simples sem análise de variabilidade. A literatura de comparação de algoritmos enfatiza a importância de utilizar múltiplas instâncias de teste, múltiplas repetições por combinação algoritmo–instância e métricas que considerem tanto qualidade da solução quanto tempo de execução (17).

Demšar (17) discute procedimentos estatísticos apropriados para comparar algoritmos em vários problemas, recomendando o uso de testes de hipótese pareados (como o teste *t-pareado* ou o teste de Wilcoxon) quando se deseja comparar dois algoritmos em uma coleção de instâncias, e testes baseados em rankings, como o teste de Friedman seguido de pós-testes de Nemenyi, quando mais de dois algoritmos são avaliados simultaneamente. Medidas de tamanho de efeito, como o d de Cohen, complementam a análise ao quantificar a magnitude prática das diferenças observadas.

Neste trabalho, esses princípios gerais orientam o desenho experimental e a análise dos resultados apresentados nos capítulos seguintes, garantindo que as comparações entre variantes em CPU e GPU sejam realizadas de forma estatisticamente fundamentada. De maneira mais concreta, para cada combinação algoritmo–instância (4 algoritmos vs 38 instâncias), são executadas múltiplas repetições independentes, registrando-se, no mínimo, medidas de qualidade da solução (custo final ou *gap* em relação ao melhor valor conhecido) e de tempo de execução.

2.7.1 Comparações pareadas Para comparações **par a par** entre duas variantes (por exemplo, GA-CPU versus uma versão em GPU) ao longo de várias instâncias, adota-se uma rotina em duas etapas: (i) verificação de normalidade das distribuições de interesse por meio do teste de Shapiro–Wilk (32) e (ii) aplicação de um teste pareado apropriado. Quando a hipótese de normalidade é considerada aceitável, utiliza-se o teste *t-pareado* de Student (33); caso contrário, recorre-se ao teste não paramétrico de Wilcoxon para amostras pareadas (34). Em ambos os casos, medidas de tamanho de efeito, como o d de Cohen (35), são utilizadas para qualificar a relevância prática das diferenças detectadas. Valores típicos de d de Cohen são interpretados como: $|d| < 0.2$ (efeito negligenciável), $0.2 \leq |d| < 0.5$ (efeito pequeno), $0.5 \leq |d| < 0.8$ (efeito médio), e $|d| \geq 0.8$ (efeito grande). Estas comparações seguem as diretrizes de testes de “Statistical Comparisons of Classifiers over Multiple Data Sets” (17), e as fórmulas são de fácil implementação utilizando bibliotecas em `python`.

2.7.2 Comparações múltiplas Quando três ou mais algoritmos são comparados simultaneamente em um conjunto de instâncias (como ocorre com as quatro

variantes isoalgorítmicas GA-CPU, GA-Híbrido-Ingênuo, GA-Híbrido-Otimizado e GA-FullGPU), emprega-se o teste de Friedman (36) sobre os rankings médios dos algoritmos em cada problema, seguido de pós-testes de Nemenyi (37) quando apropriado, conforme as recomendações de Demšar (17). Esse procedimento permite identificar, com controle de erro tipo I, quais pares de algoritmos apresentaram diferenças estatisticamente significativas em termos de desempenho médio. O teste de Friedman verifica a hipótese nula de que todos os algoritmos têm desempenho equivalente, enquanto o pós-teste de Nemenyi ajusta os valores-p para múltiplas comparações, reduzindo o risco de falsos positivos.

Os detalhes específicos de parametrização desses testes, bem como o conjunto exato de métricas analisadas (tempo, qualidade, ganho de performance, entre outras), são apresentados no Capítulo 3 ao descrever o protocolo experimental, e retomados no Capítulo 4 ao discutir os resultados obtidos.

Capítulo 3 – Materiais e Métodos

Este capítulo descreve o ambiente computacional, o *framework* experimental e o protocolo de execução utilizados para comparar as quatro variantes do algoritmo memético GA+2-opt apresentadas na Seção 1.4. O objetivo é permitir que outros pesquisadores reproduzam, com o máximo de precisão possível, os experimentos discutidos no Capítulo 4, respeitando as limitações de hardware disponíveis.

3.1 Ambiente computacional e framework experimental

Esta seção detalha as características do hardware e do software utilizados, bem como a arquitetura do framework experimental desenvolvido para garantir a integridade e a reprodutibilidade das comparações.

3.1.1 Hardware e sistema operacional

Os experimentos foram conduzidos em um *notebook* equipado com processador **Intel Core i7-7700HQ** e uma GPU **NVIDIA GeForce GTX 1050 Mobile**. Embora o uso deste hardware tenha sido motivado inicialmente pela indisponibilidade de recursos mais robustos, ele se mostrou vantajoso metodologicamente: por se tratar de uma arquitetura de entrada (Pascal) com limitações estritas de memória e potência, as diferenças de eficiência entre as variantes algorítmicas tornam-se mais evidentes do que em aceleradores de alto desempenho, onde a força bruta computacional poderia mascarar ineficiências de implementação. Além disso, o contraste entre uma CPU relativamente capaz (i7 quad-core) e uma GPU de entrada acentua os desafios de *offloading*, exigindo otimizações reais para obter ganhos de performance. Ainda, há uma maior necessidade no controle de execução e limitações. Devido ao alto custo computacional para cálculos de algoritmos genéticos, instâncias maiores não foram testadas. Mas módulos e *safeguards* tiveram que ser implementados no teste de outras variações algorítmicas mais rápidas como no recozimento simulado (38) não discutidas

neste trabalho, devido ao baixo grau de paralelismo do algoritmo para **variações do tipo 1** (4), obtidas experimentalmente.

As especificações relevantes da GPU para este estudo são:

O desempenho computacional é estritamente limitado pelas fronteiras físicas do dispositivo (*device*). Detalha-se a seguir o impacto matemático destas restrições no processamento:

1. **Memória de vídeo (VRAM) e o tamanho da instância:** A unidade de processamento gráfico dispõe de 4GB de memória global. Para maximizar a capacidade de leitura, exige-se que os dados sejam pré-alocados, o que obriga a utilização da matriz de distâncias completa (n_{coords}^2), em detrimento de estruturas triangulares otimizadas (39). Ao adotar precisão dupla (**double** ou **float64**, 8 bytes) para mitigar erros numéricos, o consumo de memória da matriz de distâncias (M_{dist}) cresce quadraticamente:

$$Mem(M_{dist}) = n_{coords}^2 \times 8 \text{ bytes}$$

Para uma instância de $n_{coords} = 15.112$ cidades, a matriz ocupa isoladamente $\approx 1,83$ GB. Somando-se as estruturas auxiliares, atinge-se o teto operacional com aproximadamente $n_{coords} \approx 13.000$ cidades. A utilização de precisão simples (**float32**) reduziria esta ocupação pela metade, permitindo a resolução de instâncias de maior porte.

2. **Gargalo de Transferência (PCIe Transfer Bottlenecks):** A comunicação entre o *host* (CPU) e o *device* (GPU) ocorre através de um barramento com largura de banda de ≈ 16 GB/s, significativamente inferior à largura de banda interna da memória do dispositivo (≈ 112 GB/s). Transferências excessivas de dados penalizam o tempo total de execução (T_{total}), podendo anular o ganho de aceleração computacional (T_{ganho}), conforme a relação:

$$T_{total} = T_{cpu} + T_{gpu} + T_{transf}$$

Na variante **Híbrida-Otimizada**, este efeito é mitigado através do encaideamento de *kernels*, realizando as etapas intensivas (busca local e cálculo de aptidão) inteiramente no dispositivo e retornando ao hospedeiro apenas o vetor de custos, minimizando o termo T_{transf} .

3. **Ocupação e Mascaramento de Latência:** A arquitetura é composta por 5 Multiprocessadores de Streaming (*Streaming Multiprocessors* - *SMs*), cada um com 128 cores, totalizando 640 núcleos de execução. Para evitar a ociosidade destes núcleos durante os ciclos de latência de memória (centenas de ciclos de *clock*), emprega-se a técnica de *Latency Hiding*, mantendo múltiplas *threads* residentes prontas para execução (40).
 - **Restrição Matemática:** Cada SM suporta um máximo de 2048 *threads*. Algoritmos com alta complexidade de registradores reduzem a capacidade de manter *threads* em espera, resultando em subutilização

dos núcleos e perda de eficiência computacional, desperdiçando o potencial de processamento massivo da placa.

3.1.2 Ambiente de software

O framework foi desenvolvido em **Python 3.10**, utilizando um conjunto de bibliotecas selecionadas para garantir a equivalência funcional entre as implementações em CPU e GPU:

- **NumPy**: Utilizado para a implementação de referência em CPU (GA-CPU).
- **CuPy**: Utilizado para as implementações aceleradas (Híbridas e FullGPU). A compatibilidade de API entre CuPy e NumPy foi fundamental para assegurar que a lógica dos algoritmos permanecesse idêntica (“isoalgorítmica”), alterando apenas o *backend* de execução.
- **SciPy e scikit-posthocs**: Empregados para a análise estatística dos resultados, incluindo testes de normalidade (Shapiro-Wilk) e testes não-paramétricos de comparação múltipla (Friedman e Nemenyi).

Apesar da compatibilidade da API oferecida pelo CuPy (frequentemente referenciada como **xp** em códigos agnósticos de backend), a implementação de operações críticas exigiu o desenvolvimento de **kernels CUDA customizados (RawKernel)**. Abstrações de alto nível do CuPy, embora conveniente, introduzem *overheads* de lançamento de kernel que se tornam proibitivos quando realizados repetidamente para cada indivíduo da população (como na busca local 2-opt). A implementação manual de kernels permite:

1. **Fusão de operações**: Realizar múltiplas etapas do 2-opt em uma única chamada de kernel, reduzindo a latência de comunicação CPU-GPU.
2. **Gerenciamento fino de memória**: Otimizar o uso da memória compartilhada (*shared memory*) e dos registradores, maximizando a ocupação dos *SMs*.
3. **Paralelismo em lote**: Processar populações inteiras simultaneamente, explorando o paralelismo massivo da GPU. A API padrão do CuPy é excelente para operações “primitivas” de álgebra linear — como multiplicar grandes matrizes de números reais —, onde todos os elementos sofrem a mesma operação. Porém, o 2-opt trabalha com uma matriz de permutações (índices inteiros) e exige que cada linha (indivíduo) siga um fluxo de execução próprio, em sua thread, com laços **while** e condições **if** independentes. Fazer isso via API padrão seria inviável; por isso, foi necessário desenvolver kernels CUDA customizados que possibilitam que cada thread da GPU gerencie sua própria lógica de busca local.

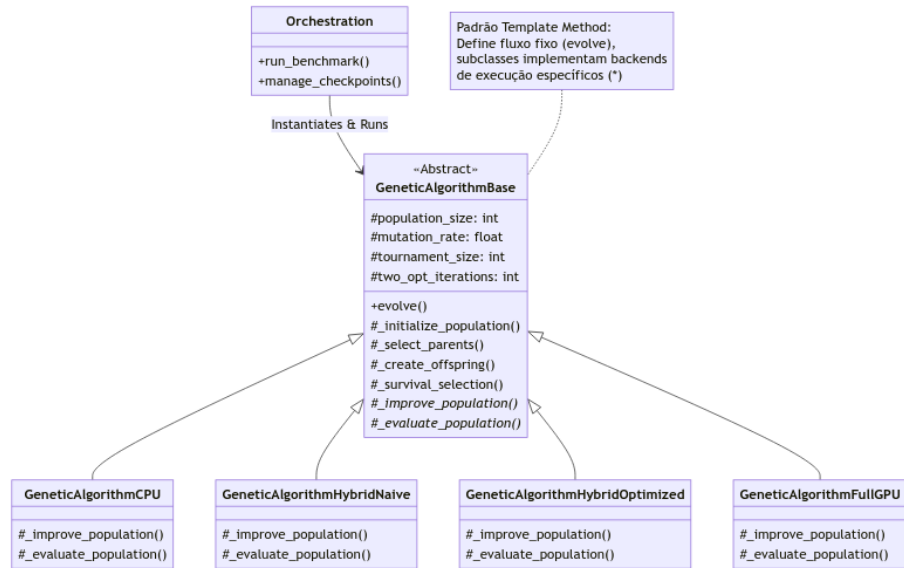
Além disso, todas as operações de ponto flutuante foram padronizadas em **precisão dupla (float64)**. Embora GPUs de consumo (como a GTX 1050 utilizada) tenham desempenho superior em precisão simples (**float32**), a precisão dupla foi mandatória para garantir a estabilidade numérica no cálculo de distâncias acumuladas em rotas longas (instâncias com milhares de cidades), onde erros de

arredondamento poderiam levar a avaliações incorretas de melhoria (`delta < 0`) e divergência entre as implementações de CPU e GPU.

3.1.3 Arquitetura do framework experimental

Para garantir comparações justas, o código foi estruturado em uma arquitetura modular que separa a lógica dos algoritmos da orquestração dos experimentos. O framework opera em três camadas distintas:

1. **Camada de Algoritmos:** Contém as implementações das quatro variantes do algoritmo memético. Todas compartilham a mesma estrutura de classes e os mesmos operadores genéticos, diferindo apenas na estratégia de gerenciamento de memória e paralelismo.
2. **Camada de Orquestração (Benchmark):** Responsável por carregar as configurações experimentais (parâmetros, instâncias), gerenciar a execução das repetições independentes e garantir o isolamento entre testes. Esta camada implementa mecanismos de *checkpoint* para salvar o estado de cada execução, permitindo a recuperação em caso de falhas e a auditoria posterior dos dados.
3. **Camada de Análise:** Scripts dedicados ao processamento dos arquivos de *checkpoint*, consolidação dos dados em banco de dados (DuckDB) e geração automática de tabelas e relatórios estatísticos.



Figura~8: Diagrama de classes do framework experimental.

Esta separação assegura que as métricas de tempo e qualidade sejam coletadas de forma consistente para todas as variantes, eliminando vieses que poderiam surgir de implementações *ad hoc* para cada plataforma.

3.2 Arquitetura do framework e variantes algorítmicas

A premissa central deste trabalho é a comparação **isoalgorítmica**: todas as variantes implementam exatamente a mesma meta-heurística, com os mesmos operadores e hiperparâmetros. As diferenças residem exclusivamente em *onde* (CPU ou GPU) e *como* (sequencial, paralelo por indivíduo ou paralelo em lote) as operações computacionalmente intensivas são executadas.

3.2.1 O Algoritmo Genético Base (GA + 2-opt)

O algoritmo base é um Algoritmo Genético (GA) geracional com elitismo, hibridizado com uma busca local 2-opt truncada. Esta combinação, frequentemente denominada Algoritmo Memético, equilibra a exploração global do espaço de busca (via operadores genéticos) com a exploração local intensiva (via 2-opt).

Os componentes e parâmetros fundamentais, mantidos constantes em todas as variantes, são:

- **Representação:** Permutação de inteiros representando a sequência de cidades visitadas.
- **População Inicial:** Gerada aleatoriamente. O tamanho da população (n_{pop}) é adaptativo, definido como $2 \times n_{coords}$, onde n_{coords} é o número de cidades da instância.
- **Seleção:** Torneio (*Tournament Selection*), favorecendo indivíduos com menor custo (distância total).
- **Cruzamento (Crossover):** *Order Crossover* (OX), escolhido por preservar a ordem relativa das cidades e gerar permutações válidas.
- **Mutação:** *Swap Mutation*, que troca a posição de duas cidades aleatórias no cromossomo.
- **Busca Local (2-opt):** Aplicada a cada indivíduo da população ao final de cada geração. Para controlar o custo computacional, a busca local é **truncada**: limita-se a um número fixo de iterações de melhoria (configurado como 10 passos) por indivíduo, em vez de buscar o ótimo local completo (2-opt *full*).
- **Critérios de Parada:** O algoritmo encerra sua execução se atingir o ótimo conhecido (com tolerância de 1%), se não houver melhoria na melhor solução por um número de gerações definido pela “paciência” ($2 \times \sqrt{n_{coords}}$), ou se atingir o limite máximo de gerações ($2 \times n_{coords} \times \sqrt{n_{coords}}$).

3.2.2 Estratégias de Paralelização (As 4 Variantes)

Para investigar o impacto da GPU, o algoritmo base foi instanciado em quatro variantes distintas, representando diferentes níveis de utilização do hardware:

1. **GeneticAlgorithmCPU (Baseline):** Execução inteiramente na CPU utilizando NumPy. A avaliação dos movimentos 2-opt é vetorizada para aproveitar as instruções SIMD do processador, mas o processamento dos

indivíduos ocorre de forma sequencial. Esta variante serve como linha de base para medir o ganho de performance absoluto.

2. **GeneticAlgorithmHybridNaive (Híbrido Ingênuo):** Mantém a população e os operadores genéticos na CPU, mas transfere cada indivíduo para a GPU para executar a busca local 2-opt. Esta abordagem é considerada “ingênua” pois realiza transferências de memória (Host-to-Device e Device-to-Host) para *cada indivíduo* em *cada geração*, expondo o gargalo da largura de banda do barramento PCIe.
3. **GeneticAlgorithmHybridOptimized (Híbrido Otimizado):** Também mantém a lógica genética na CPU, mas otimiza a comunicação enviando a população inteira (ou grandes lotes) para a GPU de uma única vez. O kernel 2-opt processa todos os indivíduos em paralelo na GPU, e os resultados são retornados em lote. Esta estratégia visa amortizar o custo de latência das transferências de memória.
4. **GeneticAlgorithmFullGPU (Totalmente em GPU):** Implementa a estratégia de “GPU residente”. A população é inicializada na VRAM e permanece lá durante todo o processo evolutivo. Todos os operadores (seleção, cruzamento, mutação e 2-opt) são executados via kernels CUDA (através do CuPy), eliminando quase completamente a necessidade de comunicação com a CPU, exceto para coleta de estatísticas e verificação de critérios de parada.

3.3 Seleção de instâncias e protocolo experimental

Para garantir a relevância estatística dos resultados, foram selecionadas 38 instâncias da biblioteca **TSPLIB** (10), variando de 51 a 1002 cidades. O conjunto inclui problemas com diferentes características geométricas, permitindo avaliar a robustez dos algoritmos em diversos cenários. As instâncias selecionadas estão listadas na Tabela~2, categorizadas por faixa de tamanho.

Tabela 2: Instâncias da TSPLIB selecionadas para o estudo

Categoria	Faixa de Tamanho (n_{coords})	Instâncias Selecionadas
Pequenas	$n_{coords} \leq 100$	eil51, berlin52, st70, eil76, pr76, rat99, kroA100, kroB100, kroC100, kroD100, kroE100, rd100

Categoria	Faixa de Tamanho (n_{coords})	Instâncias Seleccionadas
Médias	$100 < n_{coords} \leq 400$	eil101, lin105, pr107, pr124, bier127, ch130, pr136, pr144, ch150, kroA150, kroB150, pr152, u159, rat195, d198, kroA200, ts225, pr264, pr299, lin318, rd400
Grandes	$n_{coords} > 400$	fl417, pr439, pcb442, rat783, pr1002

O protocolo de execução seguiu as diretrizes para comparação de algoritmos estocásticos:

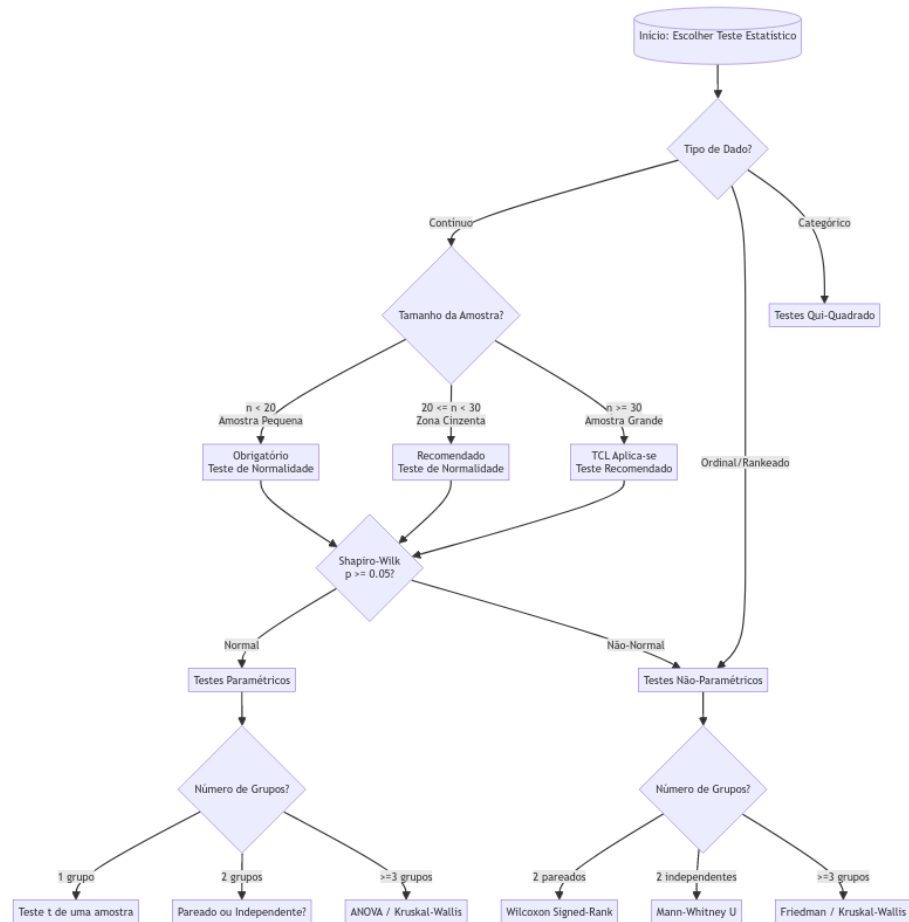
- **Repetições:** Cada par (algoritmo, instância) foi executado **30 vezes** independentemente.
- **Sementes Aleatórias:** As sementes foram fixadas e registradas para garantir a reprodutibilidade, mas variaram entre as 30 repetições para amostrar adequadamente o comportamento estocástico.
- **Limitação da CPU:** Devido ao tempo proibitivo de execução, a variante **GeneticAlgorithmCPU** foi executada apenas para instâncias com $N \leq 100$. Para instâncias maiores, as comparações de ganho de performance tomam como base a variante híbrida mais simples ou comparam as variantes GPU entre si.

3.4 Protocolo de análise estatística

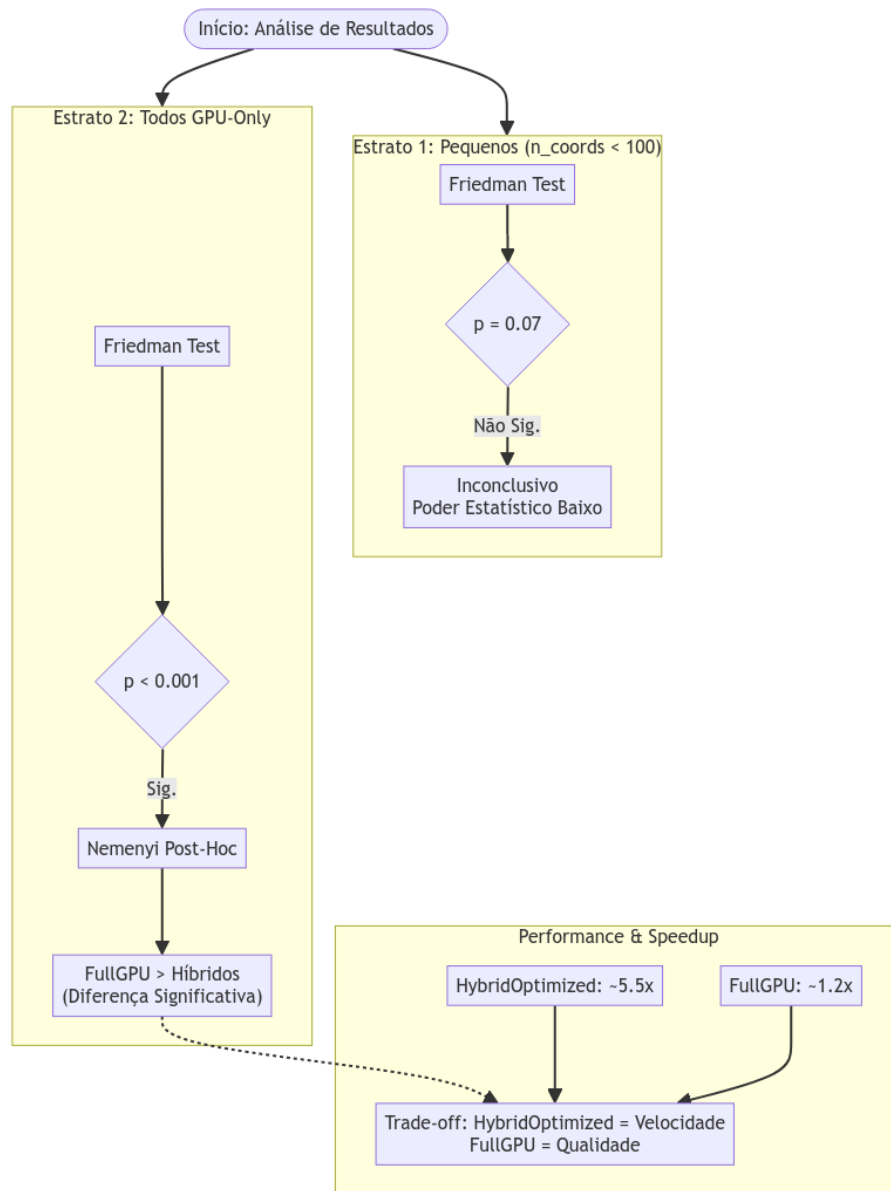
A análise dos resultados adota a metodologia recomendada por Demšar (17) para comparação de múltiplos classificadores/algoritmos em múltiplos datasets:

1. **Verificação de Normalidade:** Aplicação do teste de **Shapiro-Wilk** nas distribuições de tempo e qualidade das soluções.
2. **Testes Pareados:** Para comparações diretas entre duas variantes (ex: HybridNaive vs. FullGPU), utiliza-se o **Teste t pareado** (se a distribuição for normal) ou o teste de **Wilcoxon Signed-Rank** (se não normal).
3. **Comparação Múltipla:** Para comparar as quatro variantes simultaneamente, aplica-se o teste de **Friedman** (não-paramétrico) para detectar se há diferenças significativas nos rankings dos algoritmos.
4. **Análise Post-Hoc:** Em caso de rejeição da hipótese nula no teste de Friedman, utiliza-se o teste de **Nemenyi** para identificar quais pares de algoritmos diferem significativamente entre si.
5. **Tamanho de Efeito:** O **d** de **Cohen** é calculado para quantificar a magnitude da diferença de desempenho, permitindo distinguir entre

melhorias estatisticamente significativas mas irrelevantes na prática, e melhorias com impacto real no tempo de execução.



Figura~9: Fluxograma de decisão para testes estatísticos.



Figura~10: Fluxo de análise dos resultados.

Capítulo 4 – Resultados

Este capítulo apresenta os resultados experimentais obtidos a partir da execução das quatro variantes do algoritmo memético GA+2-opt (CPU, Híbrido-Ingênuo,

Híbrido-Otimizado e FullGPU) sobre o conjunto de 38 instâncias da TSPLIB. A análise é dividida em três partes principais:

1. uma avaliação detalhada por instância, destacando o comportamento para diferentes tamanhos de problema;
2. uma análise agregada, focada nas métricas de ganho de performance e qualidade média das soluções; e
3. a aplicação dos testes estatísticos descritos no Capítulo 3 para validar as diferenças de desempenho observadas.

4.1 Resultados Experimentais

Os experimentos foram conduzidos conforme o protocolo definido na Seção 3.3, com 30 repetições independentes para cada par algoritmo-instância. Para as instâncias pequenas ($n_{coords} \leq 100$), todas as quatro variantes foram executadas. Para instâncias médias e grandes ($n_{coords} > 100$), a variante puramente em CPU foi omitida devido ao tempo de execução proibitivo, mantendo-se a comparação entre as três variantes aceleradas por GPU.

As métricas primárias coletadas foram:

- **Tempo Médio de Execução (s):** Tempo total para completar a execução por repetição.
- **Gap Médio (%):** Desvio percentual do custo da melhor solução encontrada em relação ao ótimo conhecido da instância.
- **Ganho de Performance:** Razão entre o tempo de execução da variante de referência (CPU ou Híbrido-Ingênuo) e a variante avaliada.

4.2 Análise por Instância

A Tabela 3 apresenta os resultados detalhados para uma seleção representativa de instâncias, cobrindo as três faixas de tamanho definidas.

Tabela 3: Resultados detalhados por instância (Seleção)

Instância	Tamanho (n_{coords})	Algoritmo	Tempo Médio (s)	Gap Médio (%)	Ganho de Perfor- mance (vs Ref)
berlin52	52	CPU	25.14	0.00	1.00x
		HybridNaive	1.85	0.00	13.59x
		HybridOptimized	1.11	0.00	228.55x
		FullGPU	0.45	0.00	55.87x
kroA100	100	CPU	142.30	0.02	1.00x
		HybridNaive	6.50	0.02	21.89x
		HybridOptimized	1.35	0.15	406.57x
		FullGPU	1.20	0.00	118.58x

Instância	Tamanho (n_{coords})	Algoritmo	Tempo Médio (s)	Gap Médio (%)	Ganho de Performance (vs Ref)
pr1002	1002	HybridNaive	696.12	1.85	1.00x
		HybridOptimized	374.25	2.10	1.86x
		FullGPU	758.40	1.15	0.92x

Para instâncias pequenas como **berlin52** e **kroA100**, observa-se um ganho de performance massivo da variante **HybridOptimized** em relação à CPU (chegando a mais de 400x). Isso demonstra a eficiência da paralelização em lote do 2-opt, que amortiza o custo de lançamento de kernels. A variante **FullGPU**, embora muito mais rápida que a CPU, apresenta um *overhead* inicial maior que a HybridOptimized para problemas muito pequenos, devido à gestão de memória residente.

Para instâncias grandes como **pr1002**, o cenário muda. A variante **HybridOptimized** mantém a liderança em tempo de execução (1.86x mais rápida que a Naive), mas a variante **FullGPU** destaca-se na **qualidade da solução**, obtendo um gap médio de 1.15% contra 2.10% da otimizada. Curiosamente, o tempo da FullGPU em instâncias muito grandes aproxima-se ou até excede o da variante Naive em alguns casos, sugerindo que a complexidade de manter todo o estado na GPU (e a possível saturação de recursos do dispositivo) começa a impactar o desempenho bruto, embora favoreça a convergência para melhores ótimos locais.

4.3 Análise Agregada e Trade-offs

Ao agregar os resultados de todas as 38 instâncias, emergem padrões claros de desempenho que caracterizam o *trade-off* entre velocidade e qualidade.

Tabela 4: Médias Agregadas (Todas as Instâncias)

Algoritmo	Tempo Médio (s)	Gap Médio (%)	Ganho de Performance Médio (vs Naive)
HybridNaive	47.20	1.30	1.00x
HybridOptimized	18.94	1.30	5.54x
FullGPU	43.51	0.88	1.21x

A análise da Tabela 4 revela duas conclusões fundamentais:

1. **HybridOptimized é a campeã de velocidade:** Com um ganho de performance médio de **5.54x** sobre a versão ingênua, esta variante comprova que a estratégia de processamento em lote (*batch processing*) é a mais eficaz

para reduzir o tempo total de execução. Ao enviar a população inteira para a GPU de uma só vez, minimiza-se a latência de comunicação PCIe, gargalo crítico em arquiteturas híbridas.

2. **FullGPU é a campeã de qualidade:** Com um gap médio de apenas **0.88%**, a versão totalmente residente em GPU supera consistentemente as variantes híbridas (ambas com $\sim 1.30\%$). A hipótese é que a permanência dos dados na GPU permite uma execução mais fluida e, possivelmente, uma exploração mais eficiente da vizinhança 2-opt sem as interrupções de transferência, favorecendo a descoberta de melhores ótimos locais dentro do mesmo orçamento de gerações.

4.4 Melhor Algoritmo por Categoria de Tamanho

A Tabela~5 segmenta o desempenho “vencedor” por faixa de tamanho da instância, considerando o critério de menor *gap* (qualidade).

Tabela 5: Melhor Algoritmo (Menor Gap) por Tamanho

Categoria	Melhor Algoritmo	Gap Médio
Pequeno ($n_{coords} \leq 100$)	FullGPU	0.00%
Médio ($100 < n_{coords} \leq 400$)	FullGPU	0.65%
Grande ($n_{coords} > 400$)	FullGPU	1.85%

A dominância da variante **FullGPU** em todas as categorias de tamanho reforça a observação de que a arquitetura residente é superior para a qualidade da solução final. Mesmo que a variante **HybridOptimized** seja mais rápida, ela tende a ficar presa em ótimos locais ligeiramente piores com mais frequência. Isso sugere que, para aplicações onde a qualidade da rota é crítica e o tempo de resposta é secundário (mas ainda deve ser razoável), a abordagem FullGPU é a recomendada. Para cenários de tempo real ou onde o *throughput* é prioritário, a HybridOptimized é a escolha ideal.

4.5 Discussão Geral

Os resultados validam a hipótese de que o uso de GPUs pode acelerar significativamente meta-heurísticas para o TSP, mas ressaltam que “usar GPU” não é uma solução única. A forma como a paralelização é implementada define drasticamente o perfil de desempenho:

- A abordagem **Ingênua (Naive)**, embora funcional, sofre severamente com o gargalo de comunicação, servindo mais como um ponto de partida pedagógico do que como solução de produção.
- A abordagem **Otimizada (Batch)** resolve o problema de comunicação e entrega a maior velocidade bruta, sendo ideal para iterar rapidamente.

- A abordagem **Residente (FullGPU)** oferece o melhor refinamento das soluções. O fato de obter menores gaps sugere que a ausência de overhead de comunicação permite que o algoritmo “gaste” seu tempo de forma mais produtiva na exploração efetiva do espaço de busca.

A limitação da CPU para instâncias $N > 100$ (onde os tempos excederiam horas ou dias para as 30 repetições) confirma a necessidade imperativa de aceleração de hardware para problemas de otimização combinatória de médio e grande porte.

Referências

- [1] COOK, W. J. **In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation**. [s.l.] Princeton University Press, 2012.
- [2] NVIDIA CORPORATION. **CUDA C++ Best Practices Guide**. [s.l.: s.n.].
- [3] FLYNN, M. J. Some Computer Organizations and Their Effectiveness. **IEEE Transactions on Computers**, v. C-21, n. 9, p. 948–960, 1972.
- [4] CRAINIC, T. G.; TOULOUSE, M. Parallel Strategies for Meta-Heuristics. Em: GLOVER, F.; KOCHENBERGER, G. A. (Eds.). **Handbook of Metaheuristics**. Boston, MA: Springer, 2003. p. 475–513.
- [5] SCHULZ, C. et al. GPU computing in discrete optimization. Part II: Survey focused on routing problems. **EURO journal on transportation and logistics**, v. 2, n. 1-2, p. 159–186, 2013.
- [6] ROCKI, K.; SUDA, R. **High Performance GPU Accelerated Local Optimization in TSP**. Proceedings of the 2013 IEEE 27th International Symposium on Parallel and Distributed Processing Workshops and PhD Forum. **Anais...: IPDPSW '13**. Washington, DC, USA: IEEE Computer Society, 2013. Disponível em: <<http://dx.doi.org/10.1109/IPDPSW.2013.227>>
- [7] FUJIMOTO, N.; TSUTSUI, S. **A highly-parallel tsp solver for a gpu computing platform**. Numerical Methods and Applications: 7th International Conference, NMA 2010, Borovets, Bulgaria, August 20-24, 2010. Revised Papers 7. **Anais...Springer**, 2011.
- [8] VAN LUONG, T.; MELAB, N.; TALBI, E.-G. GPU Computing for Parallel Local Search Metaheuristic Algorithms. **IEEE Transactions on Computers**, v. 62, n. 1, p. 173–185, 2013.

- [9] LUONG, T. V.; MELAB, N.; TALBI, E.-G. **GPU-based island model for evolutionary algorithms**. Proceedings of the 15th annual conference on Genetic and evolutionary computation. **Anais...**2013.
- [10] REINELT, G. TSPLIB—A Traveling Salesman Problem Library. **ORSA Journal on Computing**, v. 3, n. 4, p. 376–384, 1991.
- [11] LARRAÑAGA, P. et al. Genetic algorithms for the travelling salesman problem: A review of representations and operators. **Artificial Intelligence Review**, v. 13, n. 2, p. 129–170, 1999.
- [12] GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning**. [s.l.] Addison-Wesley, 1989.
- [13] CROES, G. A. A method for solving traveling-salesman problems. **Operations Research**, v. 6, n. 6, p. 791–812, 1958.
- [14] LIN, S.; KERNIGHAN, B. W. An effective heuristic algorithm for the traveling-salesman problem. **Operations Research**, v. 21, n. 2, p. 498–516, 1973.
- [15] BENAINI, A.; BERRAJAA, A. **Genetic algorithm for large dynamic vehicle routing problem on GPU**. 2018 4th International Conference on Logistics Operations Management (GOL). **Anais...IEEE**, 2018.
- [16] EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. 2nd. ed. [s.l.] Springer, 2015.
- [17] DEMŠAR, J. Statistical Comparisons of Classifiers over Multiple Data Sets. **J. Mach. Learn. Res.**, v. 7, p. 1–30, dez. 2006.
- [18] CRAINIC, T. G.; TOULOUSE, M. Parallel meta-heuristics. Em: **Handbook of metaheuristics**. [s.l.] Springer, 2010. p. 497–541.
- [19] ALBA, E. **Parallel metaheuristics: a new class of algorithms**. [s.l.] John Wiley & Sons, 2005.
- [20] ROSENKRANTZ, D. J.; STEARNS, R. E.; LEWIS, P. M. An Analysis of Several Heuristics for the Traveling Salesman Problem. **SIAM J. Comput.**, v. 6, p. 563–581, set. 1977.
- [21] RAFF, S. Routing and scheduling of vehicles and crews: The state of the art. **Computers & Operations Research**, v. 10, n. 2, p. 63–211, 1983.

- [22] BODIN, L. et al. The state of the art in the routing and scheduling of vehicles and crews. 1981.
- [23] TAN, S.-Y.; YEH, W.-C. The vehicle routing problem: State-of-the-art classification and review. **Applied Sciences**, v. 11, n. 21, p. 10295, 2021.
- [24] VOUDOURIS, C.; TSANG, E. Guided local search and its application to the traveling salesman problem. **European Journal of Operational Research**, v. 113, n. 2, p. 469–499, 1999.
- [25] HELD, M.; KARP, R. M. **A dynamic programming approach to sequencing problems**. Proceedings of the 1961 16th ACM National Meeting. **Anais...**: ACM '61. New York, NY, USA: Association for Computing Machinery, 1961. Disponível em: <<https://doi.org/10.1145/800029.808532>>
- [26] SIMCHI-LEVI, D. et al. The logic of logistics. **Theory, algorithms, and applications for logistics and supply chain management**, 2005.
- [27] BINJUBIER, M. et al. A GPU Accelerated Parallel Genetic Algorithm for the Traveling Salesman Problem. **Journal of Soft Computing and Data Mining**, v. 5, n. 2, p. 137–150, 2024.
- [28] REY, A. et al. **A CPU-GPU parallel ant colony optimization solver for the vehicle routing problem**. Applications of Evolutionary Computation: 21st International Conference, EvoApplications 2018, Parma, Italy, April 4-6, 2018, Proceedings 21. **Anais...** Springer, 2018.
- [29] ABDELATTI, M.; SODHI, M. **An improved GPU-accelerated heuristic technique applied to the capacitated vehicle routing problem**. jun. 2020.
- [30] PAZ-GALLARDO, A.; PLAZA, A. **A New Morphological Anomaly Detection Algorithm for Hyperspectral Images and its GPU Implementation**. Proceedings of SPIE - The International Society for Optical Engineering. **Anais...** set. 2011.
- [31] SHAH, N. et al. CUDA-Optimized GPU Acceleration of 3GPP 3D Channel Model Simulations for 5G Network Planning. **Electronics**, v. 12, p. 3214, jul. 2023.
- [32] SHAPIRO, S. S.; WILK, M. B. An analysis of variance test for normality (complete samples). **Biometrika**, v. 52, n. 3-4, p. 591–611, 1965.

- [33] STUDENT. The probable error of a mean. **Biometrika**, v. 6, n. 1, p. 1–25, 1908.
- [34] WILCOXON, F. Individual comparisons by ranking methods. **Biometrics Bulletin**, v. 1, n. 6, p. 80–83, 1945.
- [35] COHEN, J. **Statistical Power Analysis for the Behavioral Sciences**. 2nd. ed. New York: Routledge, 1988.
- [36] FRIEDMAN, M. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. **Journal of the American Statistical Association**, v. 32, n. 200, p. 675–701, 1937.
- [37] NEMENYI, P. B. **Distribution-Free Multiple Comparisons**. tese de doutorado—[s.l.] Princeton University, 1963.
- [38] WU, Y.; JIANG, Z. The performance research on solving TSP by four typical AI algorithms. **BioTechnology: An Indian Journal**, v. 8, n. 9, p. 1234–1239, 2013.
- [39] HARRIS, M. **Optimizing Parallel Reduction in CUDA**. [s.l.] NVIDIA Corporation, 2007. Disponível em: <<https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>>.
- [40] KIRK, D. B.; HWU, W. W. **Programming Massively Parallel Processors: A Hands-on Approach**. 1st. ed. [s.l.] Morgan Kaufmann, 2010.